

Technologies web dynamiques avec JavaScript avancé



G. Cherradi 2025

https://www.researchgate.net/profile/Ghyzlane_Cherradi



PLAN

- Introduction (Web application, JavaScript avancé)
- Manipulation avancée du DOM et des événements
- Modules ES6 et Structuration du Code
- Programmation asynchrone (Callbacks, Promises, Async/Await)
- Programmation Orientée Objet (POO) en JavaScript
- Manipulation de Données avec JSON/XML et APIs
- API Web, AJAX et Fetch
- Introduction à React.js (Front-end moderne)
- Introduction à Nest.js (Back-end JavaScript)
- Optimisation et Sécurité



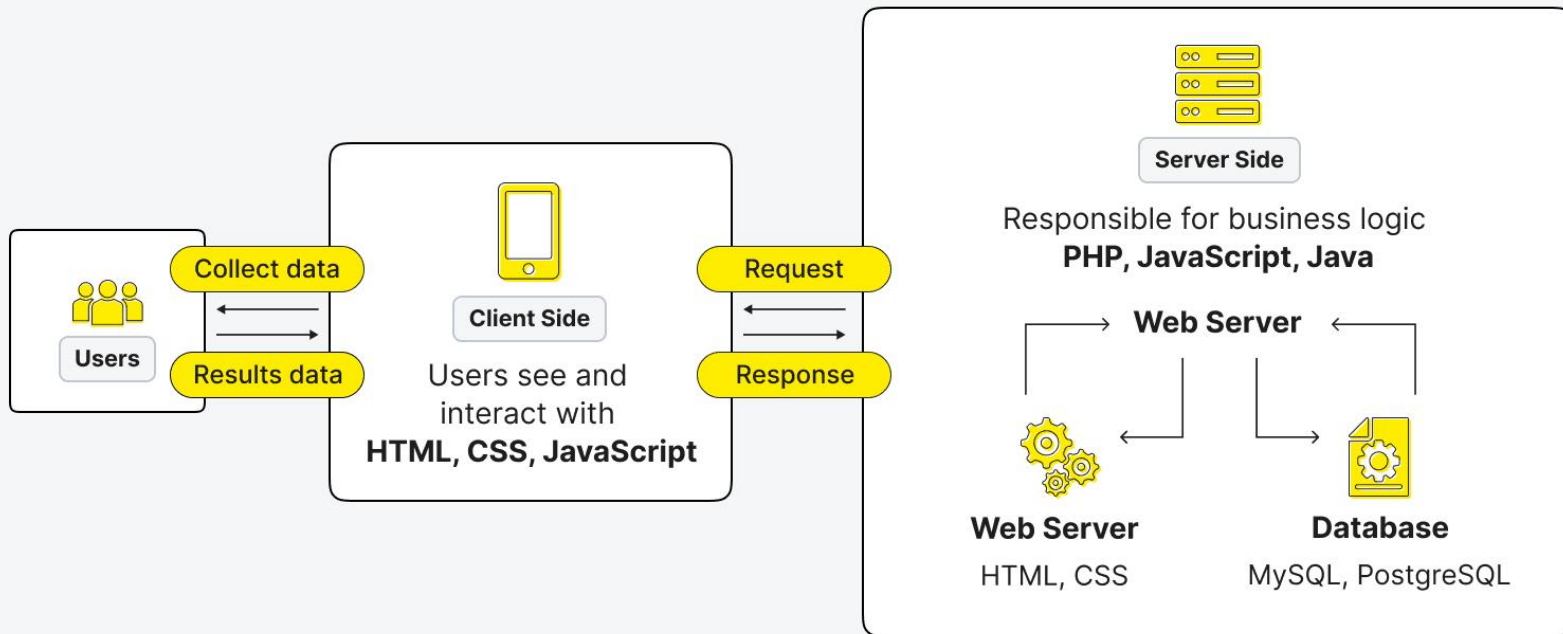
Knowledge Check (Quiz) + Badges



<https://www.youracclaim.com/organizations/ibm/badges>

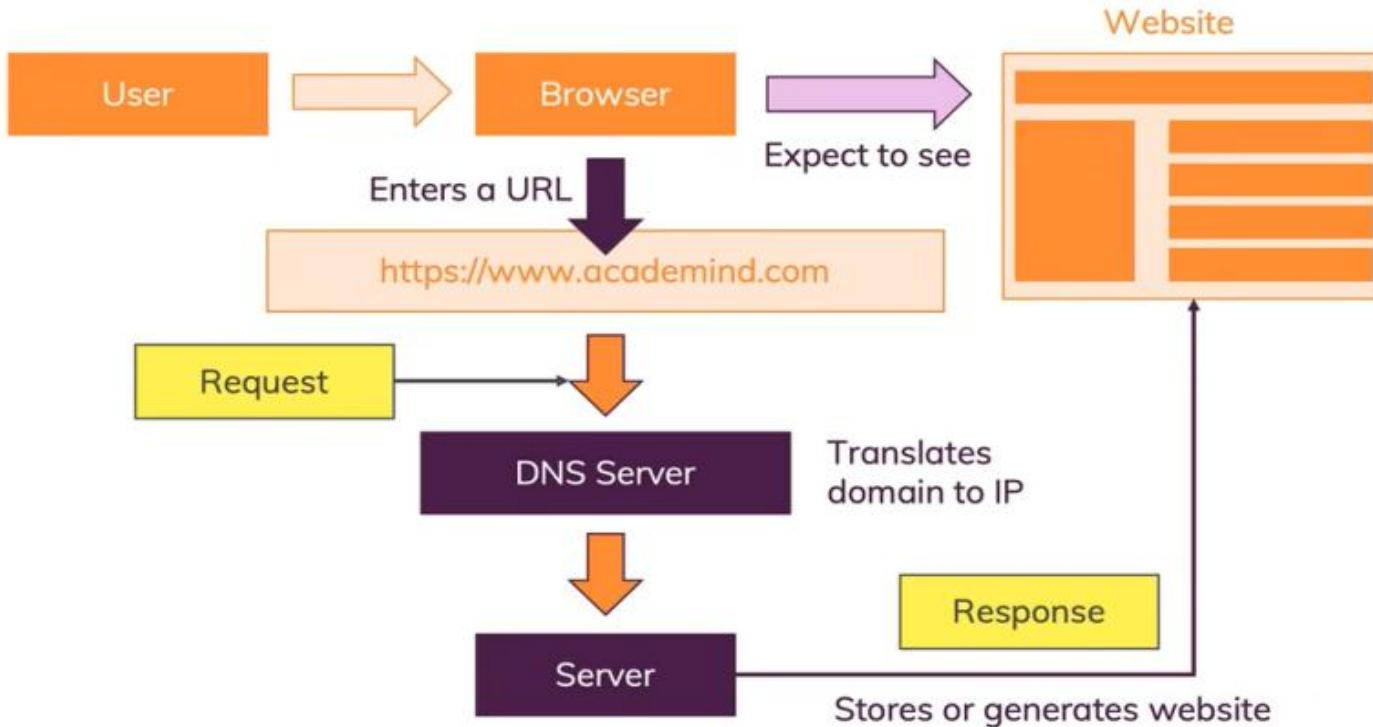
Introduction au langage JavaScript

System Architecture for a Web Application



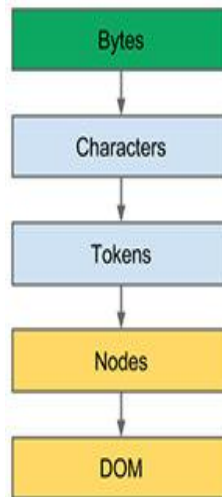
Introduction au langage JavaScript

How The Web Works



Introduction au langage JavaScript

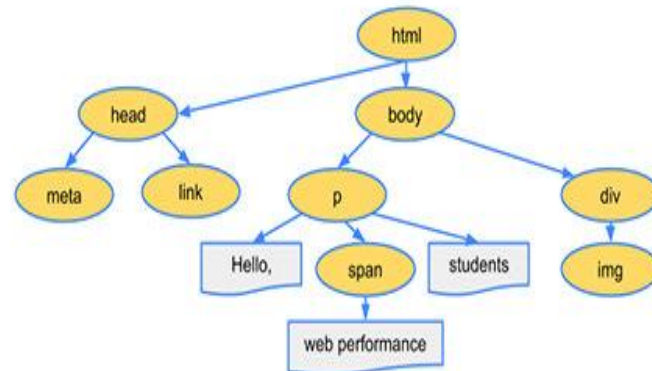
1. Le navigateur télécharge le fichier HTML sous forme de bytes
2. À l'aide de l'encodage (UTF-8), le navigateur transforme les octets en caractères lisibles.
3. Le navigateur lit le texte caractère par caractère, et il le découpe en tokens (Un token, c'est un petit morceau significatif).
4. Le navigateur transforme chaque token en node (Un node, c'est un élément vivant en mémoire). Chaque node est un objet JavaScript interne.
5. Le navigateur organise tous les nodes en arbre hiérarchique (**DOM** : document object Model).



3C 62 6F 64 79 3E 48 65 6C 6C 6F 2C 20 3C 73 70 61 6E 3E 77 6F 72 6C 64 21 3C 2F 73 70 61 6E 3E 3C 2F 62 6F 64 79 3E

<html><head>...</head><body><p>Hello web performance...</body></html>

StartTag: html StartTag: head ... EndTag: head StartTag: body StartTag: p Hello ...

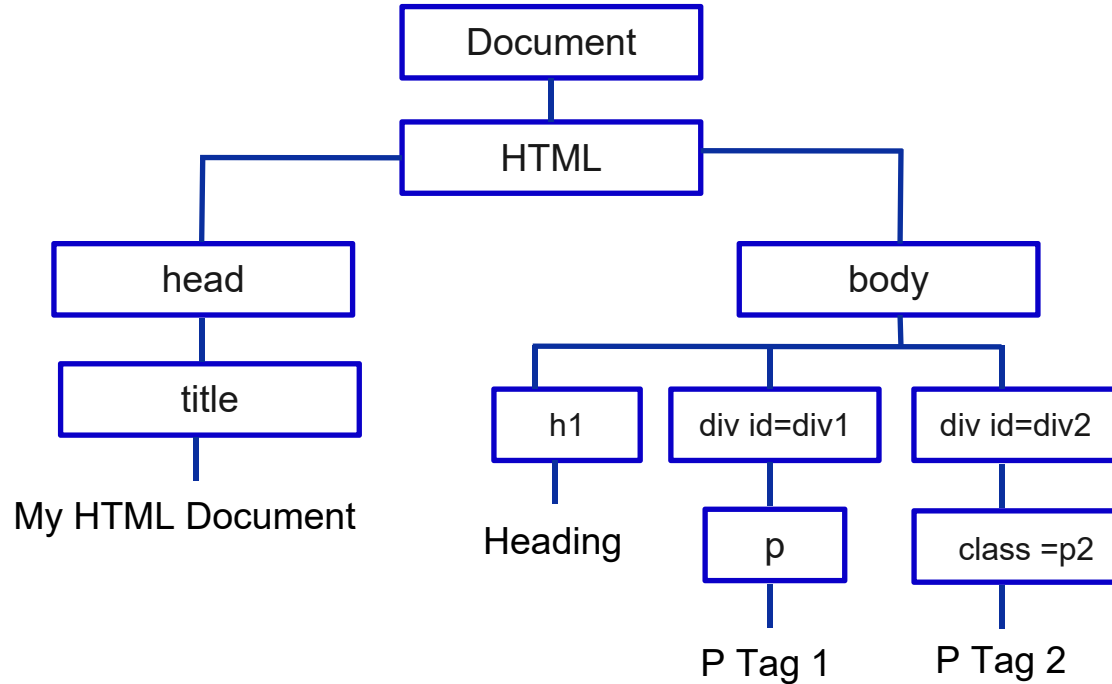


Document Object Model (DOM)

HTML Document

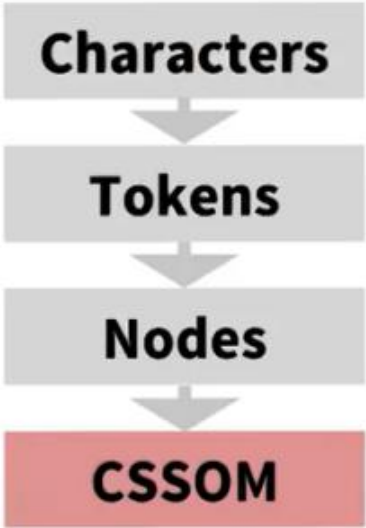
```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>My HTML Document</title>
5  </head>
6  <body>
7  |   <h1>Heading</h1>
8  |   <div id="div1">
9  | |   <p>P Tag 1</p>
10 |   </div>
11 |   <div id="div2">
12 | |   <p class="p2">P Tag 2</p>
13 |   </div>
14 </body>
15 </html>
```

Document Object Model (DOM)

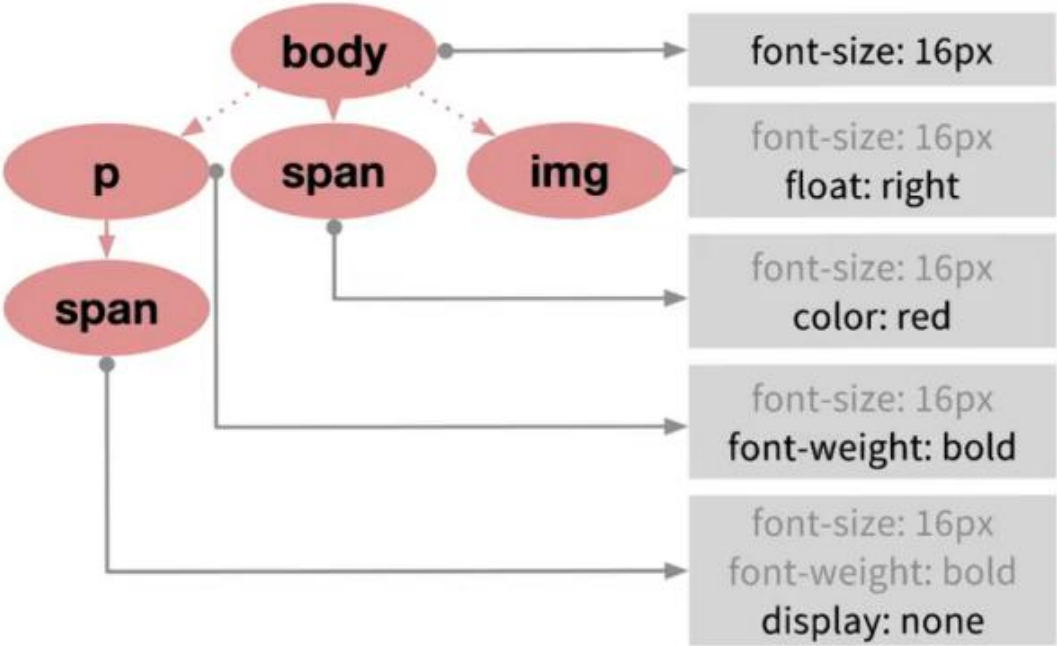


CSS Object Model (CSSOM)

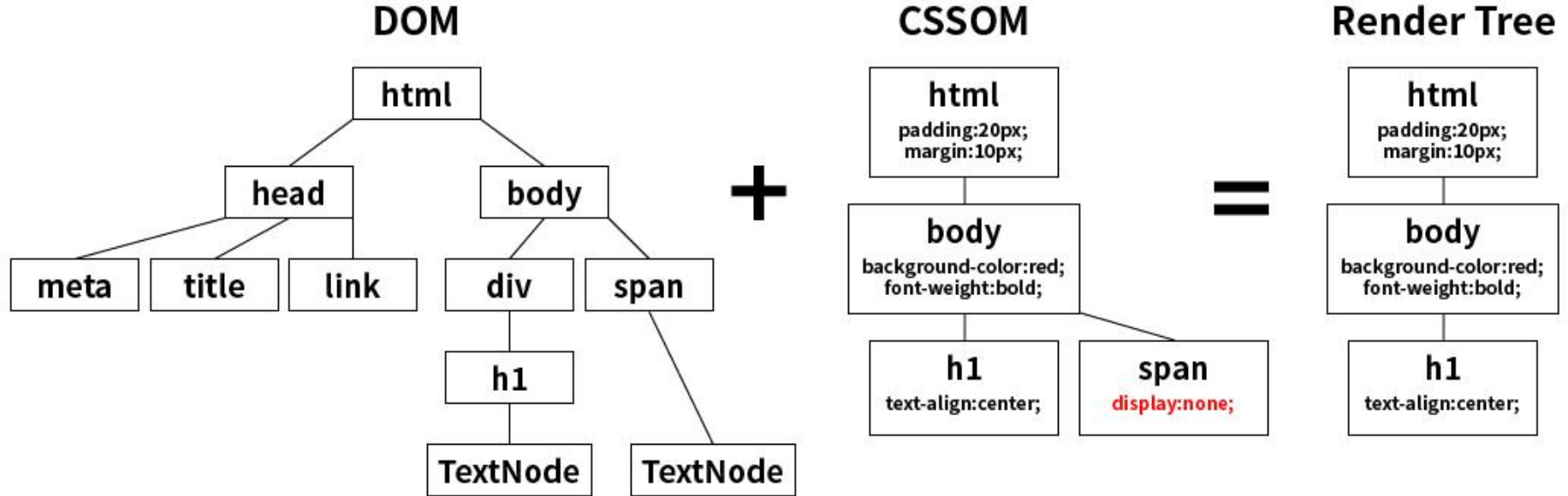
CSSOM :



```
body {font-size: 16px} p {font-weight: bold} span {color: red} p span {display: none} img {float: right}
```



Document Object Model (DOM)



Qu'est-ce que JavaScript ?

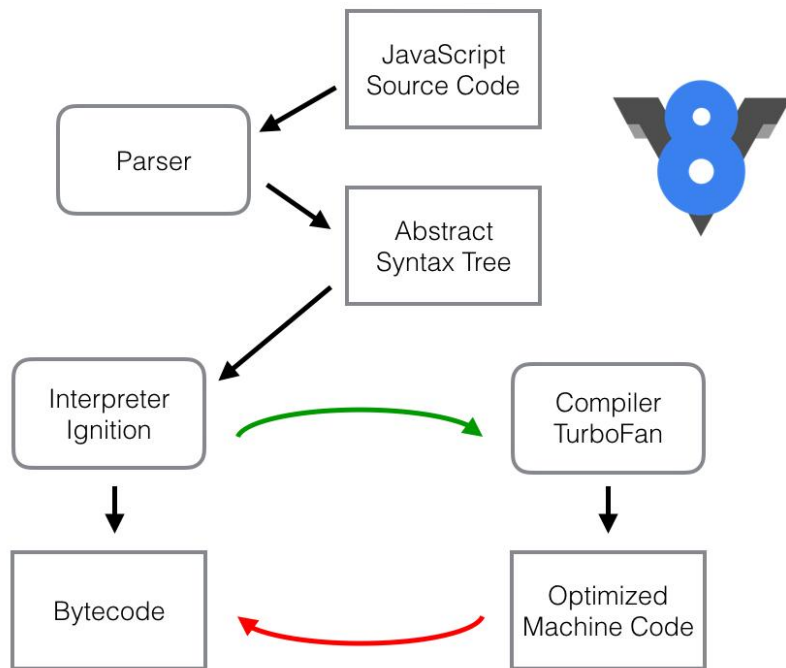
JavaScript est un **langage de programmation** utilisé pour créer du contenu dynamique pour les sites web. C'est un langage de programmation **léger, multiplateforme, mono-thread et dynamiquement typé**. JavaScript est un langage interprété qui exécute le code ligne par ligne, offrant ainsi une plus grande flexibilité.

- JavaScript est un langage mono-thread, ce qui signifie qu'il exécute une seule tâche à la fois.
- Il prend en charge à la fois le développement côté client et côté serveur
- Le type de données d'une variable est déterminé au moment de l'exécution en JavaScript.
- Polyvalent : JavaScript peut être utilisé pour une large gamme de tâches, allant de calculs simples à des applications côté serveur complexes.
- Asynchrone : JavaScript peut gérer des tâches comme la récupération de données depuis des serveurs sans bloquer l'interface utilisateur.
- Écosystème riche : Il existe de nombreuses bibliothèques et frameworks basés sur JavaScript, tels que React, Angular et Vue.js, qui rendent le développement plus rapide et plus efficace.

Qu'est-ce que JavaScript ?

JavaScript est à la fois compilé et interprété. Le **moteur V8** améliore les performances en interprétant d'abord le code, puis en compilant les fonctions fréquemment utilisées pour accélérer leur exécution. Cela rend JavaScript efficace pour les applications web modernes.

1. **Analyseur syntaxique (Parser)** : V8 commence par analyser le code source JavaScript et le transforme en un arbre syntaxique abstrait (AST).
2. **Ignition (Interpréteur)** : V8 interprète d'abord le JavaScript à l'aide d'Ignition, un interpréteur léger. Il convertit l'AST en bytecode, une représentation intermédiaire du code. Cela permet une exécution rapide sans attendre la compilation.
3. **TurboFan (Compilateur Just-In-Time - JIT)** : Pendant l'exécution, V8 identifie les fonctions fréquemment utilisées (code chaud). Ces fonctions sont compilées en code machine hautement optimisé grâce à TurboFan. Cela accélère l'exécution en supprimant les opérations inutiles.
4. **Garbage Collector (Gestion de mémoire)** : V8 possède un Garbage Collector avancé qui libère automatiquement la mémoire des objets inutilisés.



Exemple de code JavaScript et son exécution dans V8

```
function sum(a, b) {  
  return a + b;  
}  
console.log(sum(2, 3));
```

V8 lit le **code source** et le transforme en **Abstract Syntax Tree (AST)**, une représentation arborescente du code.

```
{  
  "type": "FunctionDeclaration",  
  "name": "sum",  
  "params": ["a", "b"],  
  "body": {  
    "type": "ReturnStatement",  
    "expression": {  
      "type": "BinaryExpression",  
      "operator": "+",  
      "left": "a",  
      "right": "b"}}}  
}
```

- Si la fonction `sum(2, 3)` est appelée une seule fois, elle restera en bytecode.
- Ignition, l'interpréteur de V8, prend l'AST et le convertit en bytecode.

```
mov eax, [rbp-8] // Charge `a` dans un registre  
add eax, [rbp-12] // Ajoute `b`  
ret // Retourne le résultat
```

- Si elle est appelée souvent, V8 détecte qu'elle est "chaude" (hot code) et l'envoie à TurboFan.

L'optimisation inclut des techniques comme **inlining** (remplacement d'un appel de fonction par son corps directement).

```
LdaNamedProperty a  
LdaNamedProperty b  
Add  
Return
```

Les bases du JavaScript : JavaScript interne

JavaScript interne: Le JavaScript peut être inséré directement dans un document HTML à l'aide de la balise `<script>`. il est généralement inséré soit dans la section `<head>`, soit dans `<body>` du document.

```
<!DOCTYPE html>
<html>
<head></head>
<body>
<h2 style="color: blue;">
JavaScript interne : Code JS placé dans le code HTML
</h2>
<h3 id="demo" style="color: green;"> regardez la console </h3>

<script>
  // Ceci est un commentaire sur une seule ligne
  /* Ceci est un commentaire
    sur plusieurs lignes */

  console.log("Bonjour ILSI"); // Affiche un message dans la console du navigateur
</script>

</body>
</html>
```

Les bases du JavaScript : JavaScript externe

External JavaScript : Pour les projets de grande envergure ou lorsque vous réutilisez des scripts sur plusieurs fichiers HTML, vous pouvez placer votre code JavaScript dans un fichier externe avec l'extension .js. Ce fichier est ensuite lié à votre document HTML en utilisant l'attribut src à l'intérieur d'une balise <script>.

Avantages :

- **Temps de chargement plus rapides :** Les fichiers JavaScript externes sont mis en cache par le navigateur, ce qui évite de les recharger à chaque visite.
- **Meilleure lisibilité et maintenance :** Séparer le HTML et le JavaScript rend les deux plus faciles à lire et à maintenir.
- **Séparation des responsabilités :** En isolant la structure (HTML) du comportement (JavaScript), le code devient plus propre, mieux organisé et plus modulaire.
- **Réutilisation du code :** Un seul fichier JavaScript externe peut être utilisé avec plusieurs pages HTML. Cela réduit les duplications et simplifie les mises à jour.

```
// fichier index.html
<!DOCTYPE html>
<html>
<head></head>
<body>
<h2>JavaScript externe : Code JS dans un fichier séparé
</h2>
<h3 id="demo" style="color: green;"> regardez la console
</h3>
<!-- Lien vers le fichier JavaScript externe -->
<script src="demo1.js"></script>
</body>
</html>
```

```
// fichier demo1.js

console.log("Bonjour ILSI");
```

Les bases du JavaScript : Versions de JavaScript (ECMAScript)

Aperçu des principales versions d'ECMAScript, de leurs années de sortie et des fonctionnalités clés qu'elles ont introduites.

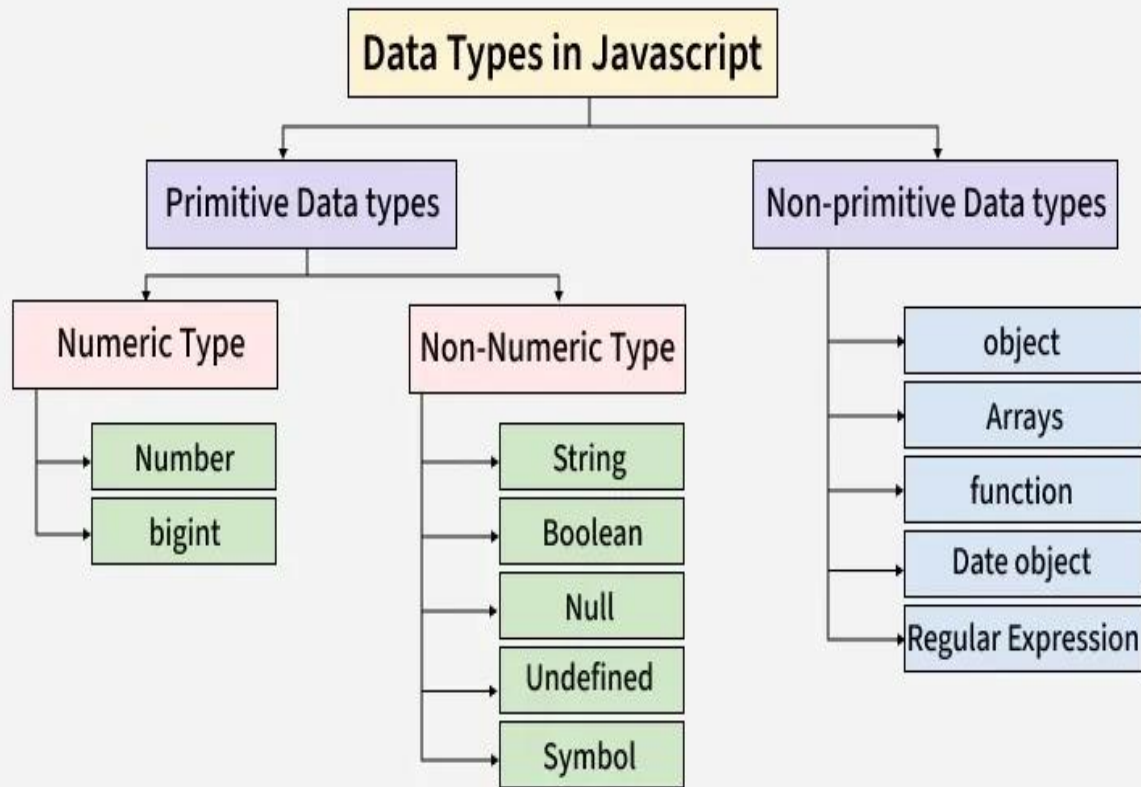
Version	Année	Fonctionnalités clés
ES5	2009	mode strict, prise en charge de JSON, getters/setters
ES6	2015	let / const, classes, fonctions fléchées
ES7 à ES13	2016 – 2022	async/await, BigInt, opérateur d'enchaînement optionnel (?.)
ES14	2023	toSorted, findLast, blocs statiques

Les bases du JavaScript : Data Types

JavaScript propose plusieurs types de données, que l'on peut regrouper en deux grandes catégories : les types primitifs et les types non primitifs.

Un type de données répond à ces questions :

- **Nature** : C'est quoi ? (Texte, nombre, booléen...)
- **Stockage** : Comment c'est mémorisé ? (Pile vs Tas, octets...)
- **Usage** : Qu'est-ce qu'on peut en faire ? (Calculs, méthodes, accès...)



Les bases du JavaScript : Data Types

Les types de données **primitifs** représentent des valeurs simples et ne peuvent pas être modifiés (immuables).

1. Number : représente les valeurs numériques (entiers et décimales)

```
let n = 42; let pi = 3.14;
```

2. String : représente du texte entouré de guillemets simples ou doubles

```
let s = "Bonjour tout le monde !";
```

3. Boolean : représente une valeur logique (true ou false)

```
let bool = true;
```

4. Undefined : Variable déclarée mais sans valeur assignée.

```
let a; console.log(a); // affiche undefined
```

5. Null — Valeur intentionnellement vide

```
let data = null;
```


Les bases du JavaScript : Data Types

Types non primitifs en JavaScript : Les types non primitifs sont des objets. Ils permettent de stocker des collections de données ou des entités plus complexes.

Object — Représente des paires clé-valeur : Un objet permet de regrouper plusieurs informations sous une même structure.

```
let obj = {  
  name: "ALI",  
  age: 25  
};
```

Array — Représente une liste de valeurs

```
let a = ["red", "green", "blue"];
```

Function — Représente un bloc de code réutilisable

```
function fun() {  
  console.log("ILISI 2025");  
}
```

Les bases du JavaScript : Les variables

Une variable est un espace de stockage utilisé pour conserver en mémoire différents types de données. Elle peut ensuite être exploitée dans les scripts. Le nom d'une variable peut inclure uniquement des caractères alphanumériques, le symbole \$ (**dollar**) et le caractère _ (**underscore**). Cependant, elle ne doit pas commencer par un chiffre ni porter le nom d'une fonction existante en JavaScript. Pour créer une variable, on la déclare et on lui attribue une valeur :

En JavaScript, les variables sont déclarées en utilisant les mots-clés **var**, **let** ou **const**.

- **Var** : Le mot-clé var est utilisé pour déclarer une variable. Il a une portée globale ou une portée fonctionnelle.

```
var n = 5;  
console.log(n);  
var n = 20; // réaffectation autorisée  
console.log(n);
```

- **let** : Le mot-clé let introduit dans ES6, a une portée de bloc et ne peut pas être redéclaré dans la même portée.

```
let n = 10;  
n = 20; // La valeur peut être mise à jour  
// let n = 15; // Ne peut pas être redéclarée dans  
// la même portée  
console.log(n);
```

- **const** : Le mot-clé const déclare des variables qui ne peuvent pas être réaffectées. Il a également une portée de bloc.

```
const n = 100;  
// n = 200; Ceci provoquera une erreur  
console.log(n);
```

Les bases du JavaScript : Types de données

JavaScript prend en charge divers types de données, qui peuvent être largement classés en types primitifs et non primitifs:

Une variable peut être de type numérique, mais aussi une chaîne de caractères :

```
var text = 'J\'écris mon texte ici'; /* Avec des apostrophes, le \ sert à  
échapper une apostrophe intégrée dans le texte, pour ne pas que Javascript  
pense que le texte s'arrête là.*/
```

Une variable peut enfin être de type booléen (boolean), avec deux états possibles :
vrai ou faux (true ou false).

Les opérateurs arithmétiques :

On peut utiliser 5 opérateurs arithmétiques : l'addition (+), la soustraction (-), la multiplication (*), la division (/) et le modulo (%). Le modulo est le reste d'une division. Par exemple : 10 : 3 qui équivaut à :

Valeur vs Référence (Types primitifs vs Objets)

```
// ---EXEMPLE DES PRIMITIFS (copie par valeur) ----
```

```
let name = 'AHMED';  
let newName = name; // copie de la valeur 'Mike'  
name = 'ALI'; // modification de la variable originale  
// name vaut maintenant "AHMED"  
// newName vaut toujours "ALI"
```

```
// ----- EXEMPLE DES OBJETS (copie par référence) -----
```

```
let person = {  
  name: 'MONA',  
  age : 24,  
  isEmployed : false  
};  
let newPerson = person; // copie de la référence ( pas duplication d'objet)  
person.name = 'ASMAA'; // modification de l'objet en mémoire  
// newPerson.name vaut aussi "ASMAA"
```

Stack

newPerson= Ref#101

person= Ref#101

name ='ALI'

newname ='AHMED'

~~name ='AHMED'~~

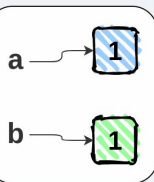
Heap

name: 'MONA'
age : 24
isEmployed: false

Ref#101

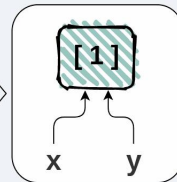
Values

```
let a = 1;  
let b = a;
```



References

```
let x = [1];  
let y = x;
```



Les bases du JavaScript : Les opérateurs arithmétiques

- Les opérateurs arithmétiques :

On peut utiliser 5 opérateurs arithmétiques : l'addition (+), la soustraction (-), la multiplication (*), la division (/) et le modulo (%). Le modulo est le reste d'une division. Par exemple : 10 : 3 qui équivaut à :

```
var number1 = 3,  
    number2 = 2, result;  
result = number1 * number2;  
alert(result);  
//Affiche : « 6 »
```

```
var number = 3;  
number = number + 5;  
alert(number);  
//Affiche : « 8 »
```

```
var number = 3;  
number += 5;  
alert(number);  
//Affiche : « 8 »
```

- **La concaténation** : Une concaténation consiste à ajouter une chaîne de caractères à la fin d'une autre, comme dans cet exemple :

```
var hi = 'Bonjour ', name = 'ILISI';  
var result = hi + name;  
alert(result);  
// Affiche : « Bonjour ILISI »
```

```
var text = 'Bonjour ';  
text += 'ilisi';  
alert(text);  
// Affiche « Bonjour ilisi ».
```

On peut convertir un nombre en chaîne de caractères avec l'astuce suivante :

```
var text, number1 = 4, number2 = 2;  
text = number1 + '' + number2; /* on ajoute une chaîne de caractères vide  
entre les deux nombres pour permettre une concaténation sans calcul */  
alert(text); // Affiche : « 42 »
```

Les bases du JavaScript : Les opérateurs de comparaison

Les huit opérateurs de comparaison

Il y en a 8 :

`==` : égal à

`!=` : différent de

`===` : contenu et type de variable égal à

`!==` : contenu ou type de variable différent de

`>` supérieur à

`>=` supérieur ou égal à

`<` inférieur à

`<=` inférieur ou égal à

Les bases du JavaScript : Data Types

Exploration des types de données et des variables en JavaScript : Comprendre les expressions courantes

```
console.log(null === undefined)
```



```
false
```

```
console.log(5 > 3 > 2)
```



```
false
```

```
console.log([ ] === [ ])
```



```
false
```

```
console.log("10" < "9")
```



```
true
```

```
console.log("ALI"/5);
```



```
NaN
```

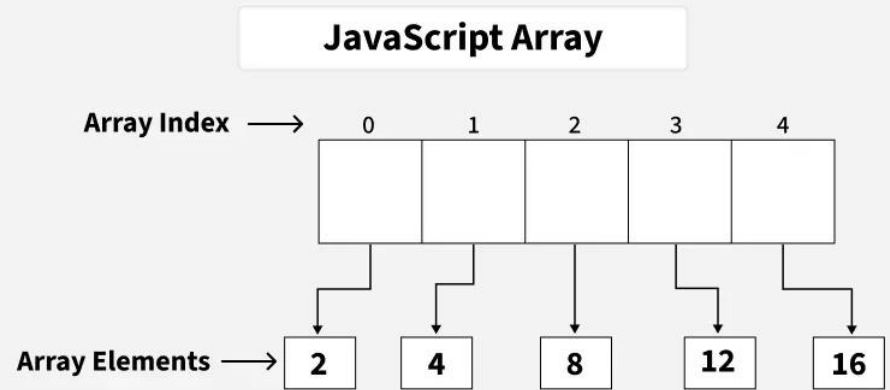
```
console.log(Infinity > 1000)
```



```
true
```

Les bases du JavaScript : JavaScript Arrays

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Accessing Array Elements
console.log(a[0]); // first element
// Accessing Last Array Elements
console.log(a[a.length - 1]);
// Add Element to the end of Array
a.push("Node.js");
// Add Element to the beginning
a.unshift("Web Development");
console.log(a);
// Removes and returns the last element
let lst = a.pop();
console.log("After Removing the last: " + a);
// Removes and returns the first element
let fst = a.shift();
console.log("After Removing the First: " + a);
// Removes 2 elements starting from index 1
a.splice(1, 2);
console.log("After Removing 2 elements starting from index 1: " + a);
```



Les bases du JavaScript : JavaScript Arrays

JavaScript Array Methods



```
.concat() .find()
.filter() .push()
.pop() .reverse()
.slice() .map()
.unshift() .splice()
.shift() .join()
.sort() .toString()
```

Méthode	Description	Exemple
concat()	Combine deux tableaux et retourne un nouveau tableau.	<code>[1,2].concat([3,4]) // [1,2,3,4]</code>
filter()	Retourne un tableau contenant uniquement les éléments qui passent un test.	<code>[1,2,3,4].filter(n => n > 2) // [3,4]</code>
pop()	Supprime et retourne le dernier élément.	<code>let a=[1,2]; a.pop(); // 2</code>
slice()	Retourne une partie du tableau sans le modifier.	<code>[1,2,3].slice(0,2) // [1,2]</code>
unshift()	Ajoute un ou plusieurs éléments au début.	<code>let a=[2,3]; a.unshift(1); // [1,2,3]</code>
shift()	Supprime et retourne le premier élément.	<code>let a=[1,2]; a.shift(); // 1</code>
sort()	Trie un tableau (attention : tri alphabétique par défaut).	<code>[3,10,1].sort() // [1,10,3]</code>
find()	Retourne le premier élément qui respecte une condition.	<code>[1,2,3].find(n => n > 1) // 2</code>
push()	Ajoute un élément à la fin du tableau.	<code>let a=[1]; a.push(2); // [1,2]</code>
reverse()	Inverse l'ordre du tableau.	<code>[1,2,3].reverse() // [3,2,1]</code>
map()	Retourne un nouveau tableau après transformation de chaque élément.	<code>[1,2,3].map(n => n*2) // [2,4,6]</code>
splice()	Modifie le tableau (ajout, suppression ou remplacement).	<code>let a=[1,2,3]; a.splice(1,1) // [1,3]</code>
join()	Transforme un tableau en chaîne séparée par un séparateur.	<code>["a","b"].join("-") // "a-b"</code>
toString()	Transforme le tableau en texte (séparé par des virgules).	<code>[1,2,3].toString() // "1,2,3"</code>

Les bases du JavaScript : Le Contexte d'Exécution (Execution Context).

Tout code JavaScript s'exécute à l'intérieur d'un **contexte d'exécution**, qui est un concept abstrait servant de **conteneur**. Il contient toutes les informations sur **l'environnement d'exécution du code actuel**.

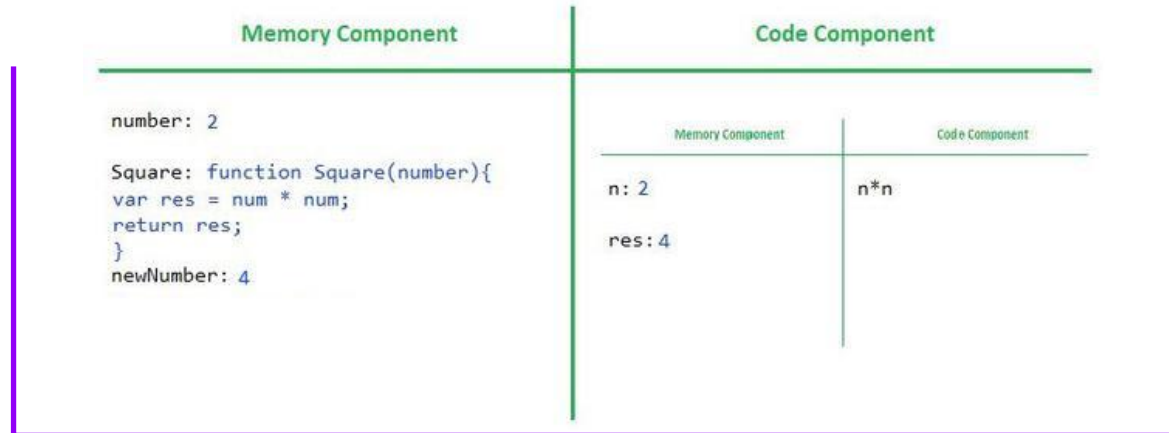
Lorsqu'un programme JavaScript s'exécute, un **contexte d'exécution est créé**, et son fonctionnement repose sur **deux phases principales**.

1. Phase d'Allocation Mémoire (Memory Allocation Phase) : JavaScript réserve de l'espace mémoire pour toutes les variables et fonctions du script.
2. Phase d'Exécution du Code (Thread d'exécution.) :

```
var number =2;
```

```
function square(num){  
  var res =num*num;  
  return res;  
}
```

```
var square1=square(number);
```



Les bases du JavaScript : fonctions

Fonctions déclarées (Function Declaration)

```
function changeLight(color) {  
    console.log(color);  
}
```

```
direBonjour(); // Fonction appelée avant sa  
définition
```

```
function direBonjour() {  
    console.log("Bonjour !");  
}
```

Fonction anonyme (Function Expression)

```
const maFonction = function() {  
    console.log("Hello !");  
};
```

```
maFonction(); // Appelle la fonction
```

- Lorsqu'on a besoin d'une fonction réutilisable.
- Quand on veut bénéficier du **hoisting** (la fonction peut être appelée avant sa déclaration).

Le hoisting est un comportement en JavaScript où **les déclarations de variables et de fonctions sont déplacées en haut de leur portée** avant l'exécution du code. Cela signifie que vous pouvez utiliser une fonction ou une variable **avant même de la déclarer** dans votre code.

- Lorsqu'on veut assigner une fonction à une variable.
- Pour des callbacks (ex: setTimeout).

Contrairement aux fonctions déclarées, les fonctions anonymes **ne sont pas hoistées** !

Les bases du JavaScript : fonctions

Fonction en tant que paramètre (Callback Function)

Une **fonction callback** est une **fonction passée en argument** à une autre fonction et qui sera **exécutée plus tard**. Cela permet d'exécuter du code **asynchrone** ou de personnaliser le comportement d'une fonction.

```
function greet(name, callback) {  
    console.log("Hello, " + name);  
    callback();  
}  
  
function sayGoodbye() {  
    console.log("Goodbye!");  
}  
  
// Appel avec callback  
greet("Alice", sayGoodbye);
```

- Programmation événementielle (DOM)
- Flexibilité et réutilisabilité
- Couplage plus faible
- **Inversion de contrôle (IoC)**

Fonctions fléchées (Arrow Functions)

```
const direBonjour = () =>  
console.log("Bonjour !");  
direBonjour(); // Affiche "Bonjour !"
```

- Les arrow functions (=>) sont une syntaxe plus courte pour écrire des fonctions en JavaScript. Elles ont été introduites avec ES6 (ECMAScript 2015) et présentent plusieurs avantages, **notamment une meilleure gestion du this.**

Les bases du JavaScript : This

- **This** dans l'espace global

Dans l'espace global, la valeur de **this** est l'objet global.

Cet objet dépend de l'environnement d'exécution :

- Navigateur → window
- Node.js → global

- **This** dans une fonction normale

La valeur de **this** dépend de la manière dont la fonction est appelée.

=> En mode non strict :

si la fonction est appelée seule → **this** est remplacé par l'objet global.

=> En mode strict :

si la fonction est appelée seule → **this** vaut undefined.

Les bases du JavaScript : This

- **This** dans les arrow functions
 - Les arrow functions n'ont pas leur propre this.
 - Elles héritent du **this** du **contexte lexical englobant**.
 - On parle de **lexical this**.
- **This** dans le DOM
 - Dans un gestionnaire d'événement avec une fonction normale :
 - **this** référence l'élément DOM qui a déclenché l'événement.
 - Avec une arrow function :
 - this n'est pas l'élément DOM (héritage lexical).

Manipulation du DOM: DOM?

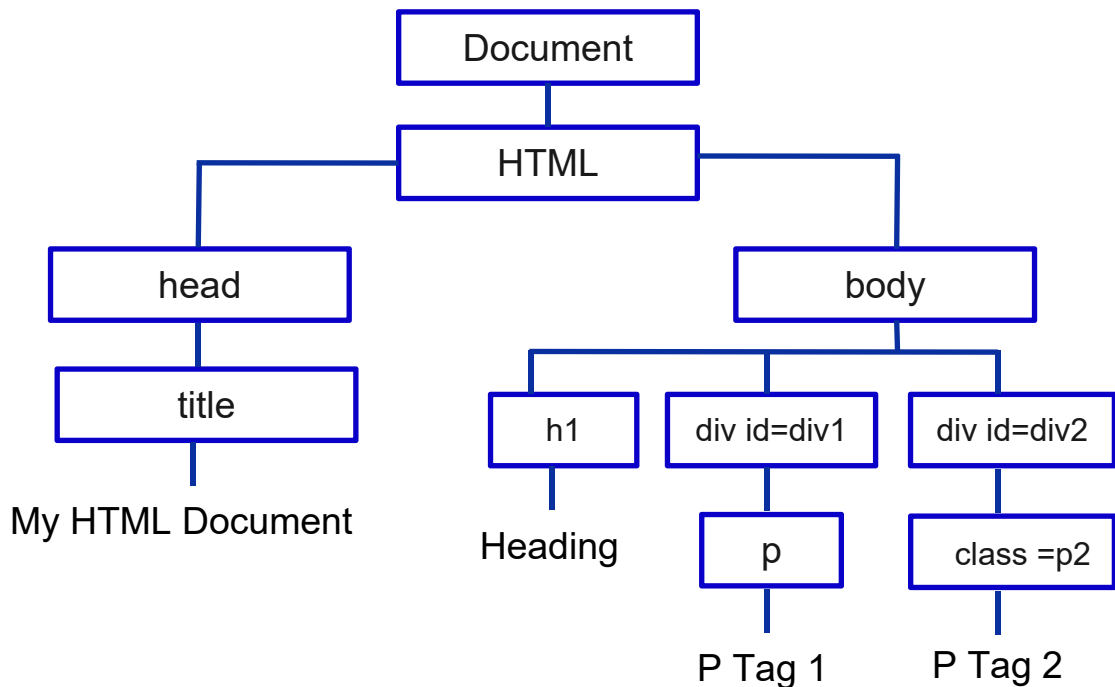
Le **DOM** (Document Object Model) est une représentation en mémoire de la page web sous forme d'un arbre d'objets.

Chaque balise = un objet

HTML Document

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>My HTML Document</title>
5  </head>
6  <body>
7  |   <h1>Heading</h1>
8  |   <div id="div1">
9  | |   <p>P Tag 1</p>
10 | </div>
11 |   <div id="div2">
12 | |   <p class="p2">P Tag 2</p>
13 | </div>
14 </body>
15 </html>
```

Document Object Model (DOM)



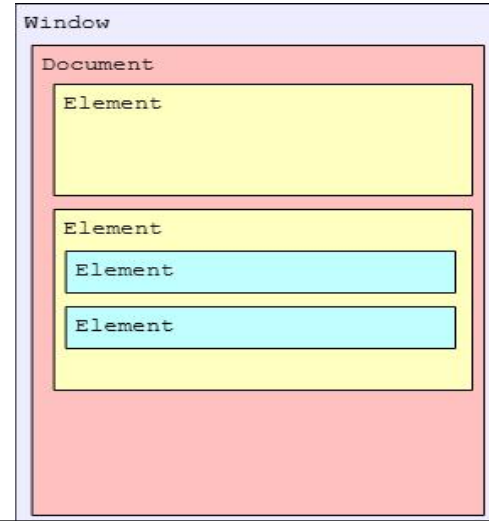
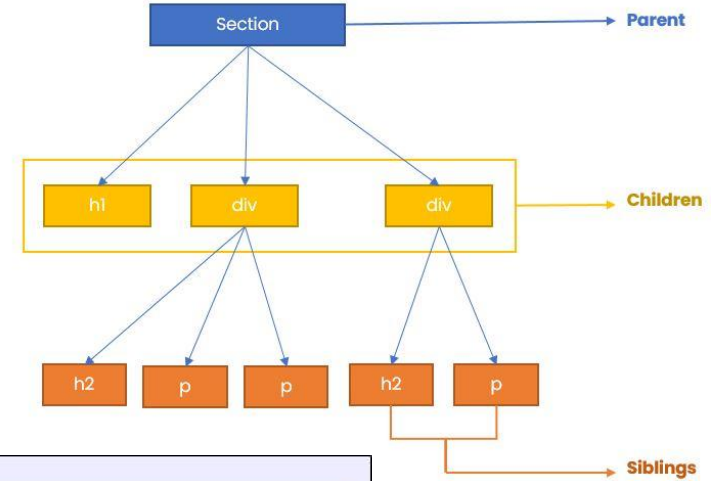
Manipulation du DOM ?

Pourquoi manipuler le DOM ?

La manipulation du DOM est essentielle pour rendre une page web dynamique et interactive. Sans cela, une page serait statique et l'utilisateur ne pourrait interagir avec elle.

Sans la manipulation du DOM, une page web resterait totalement statique :

- aucun changement de texte,
- aucun bouton interactif,
- aucun formulaire intelligent,
- aucune animation ni mise à jour automatique.



Manipulation du DOM : DOM APIs?

Le **DOM API** est un ensemble d'interfaces JavaScript permettant d'interagir avec le **DOM** (Document Object Model).

L'API DOM est indispensable pour rendre une page web interactive et dynamique. Elle fournit les méthodes nécessaires pour :

C – Create : Créer des éléments dans le DOM,

R – Read : Récupérer des éléments depuis le DOM

U – Update : Modifier des éléments existants,

D – Delete : Supprimer des éléments du DOM.



- Le **DOM** est ce que nous manipulons
- L'**API DOM** est ce qui nous permet de le manipuler.

Manipulation du DOM : Récupération d'un Élément DOM

1. **document.getElementById("id")** : Sélectionne un élément grâce à son identifiant unique.

HTML

```
<h1 id="title">Bonjour</h1>
```

Js

```
const h1 = document.getElementById("title"); // sélection par id
```

2. **document.getElementsByClassName("class")** : Sélectionne tous les éléments possédant une classe donnée.

```
<p class="text">Paragraphe 1</p>  
<p class="text">Paragraphe 2</p>
```

```
const h1 = document.getElementsByClassName("text")
```

3. **document.getElementsByTagName("tag")** : Sélectionne tous les éléments ayant une balise (tag) donnée.

```
const paragraphs = document.getElementsByTagName("p");
```

Manipulation du DOM : Récupération d'un Élément DOM

3. **document.querySelector(selector)** : Sélectionne le premier élément qui correspond au sélecteur CSS

HTML

```
<h1 id="title">Bonjour</h1>
<p class="text">Paragraphe 1</p>
<p class="text">Paragraphe 2</p>
```

Js

```
const h1 = document.querySelector("#title");    // sélection par id
const firstP = document.querySelector(".text"); // premier élément .text
const div = document.querySelector("div");      // première balise <div>
```

=> Gère tous les sélecteurs CSS : **.class**, **#id**, **tag**, **[attr]**, etc.

4. **document.querySelectorAll(selector)** : Sélectionne tous les éléments qui correspondent au sélecteur (NodeList).

```
const allParagraphs = document.querySelectorAll(".text");

allParagraphs.forEach(p => {
  console.log(p.textContent);
});
```

=> Indispensable pour modifier plusieurs éléments

Manipulation du DOM : Récupération d'un Élément DOM

Type de sélection	Exemple CSS	querySelector()
ID	#id	"#box"
Classe	.class	".item"
Balise	div	"div"
Descendant	ul li	"ul li"
Enfant direct	div > p	"div > p"
Attribut	input[type='text']	"input[type='text']"
Pseudo-class	li:first-child	"li:first-child"
Multiple	h1, h2	"h1, h2"

Manipulation du DOM : Modification d'un Élément DOM

La modification du DOM consiste à changer le **contenu**, les **attributs**, les **classes**, ou le **style** d'un élément existant dans la page. JavaScript permet de modifier dynamiquement l'apparence et le comportement d'un élément sélectionné grâce aux propriétés du DOM.

1. Modifier le contenu texte : **textContent** => Change uniquement le texte

```
const title = document.querySelector("h1");  
title.textContent = "Nouveau titre";
```

2. Modifier le contenu HTML : **innerHTML** => Interprète du HTML

```
const div = document.querySelector(".box");  
div.innerHTML = "<p>Paragraphe <strong>ajouté</strong></p>";
```

3. Modifier les attributs : **setAttribute()**

```
const img = document.querySelector("img");  
img.setAttribute("src", "photo.png");  
img.setAttribute("alt", "Nouvelle image");
```

Manipulation du DOM : Modification d'un Élément DOM

La modification du DOM consiste à changer le **contenu**, les **attributs**, les **classes**, ou le **style** d'un élément existant dans la page. JavaScript permet de modifier dynamiquement l'apparence et le comportement d'un élément sélectionné grâce aux propriétés du DOM.

4. Modifier les styles : **element.style**

```
title.style.color = "red";  
title.style.fontSize = "30px";
```

5. Modifier les classes : **classList**

```
title.classList.add("active"); // Add  
title.classList.remove("active"); // Remove  
title.classList.toggle("highlight"); // Alternner (toggle)  
title.classList.contains("active"); // Vérifier si une classe existe
```

Manipulation du DOM : Création d'un Élément DOM

La création d'éléments dans le DOM se fait en 3 étapes :

1. Créer l'élément : **document.createElement("tag")**: Crée un nouvel élément HTML en mémoire

```
const div = document.createElement("div");
```

2. Modifier ses propriétés / contenu :

```
div.textContent = "Hello World!";  
div.style.color="purple";  
div.innerText= "Hello hello!";
```

3. **document.body.appendChild(node)** : Ajoute un nœud enfant à un élément existant dans le DOM.

```
document.body.appendChild(div);
```

Manipulation du DOM : Suppression d'un Élément DOM

1. **element.remove()** : Supprime directement l'élément du DOM.

```
const p = document.querySelector("p");  
p.remove();
```

2. **parent.removeChild(child)** : Suppression via le parent

```
const list = document.querySelector("ul");  
const firstItem = list.firstElementChild;  
  
list.removeChild(firstItem);
```

3. **innerHTML = ""** : Vider complètement un élément => Permet de supprimer tout le contenu d'un élément.

```
const container = document.querySelector(".container");  
container.innerHTML = "";
```


Manipulation du DOM : Gestion des événements dans le DOM

JavaScript nous permet de gérer des événements tels que les **clics**, les **frappes** au clavier, les **mouvements** de la souris, etc.

1. Ajouter un écouteur d'événement :

```
document.getElementById("btn").addEventListener("click", function() {  
    alert("Button Clicked!");  
});
```

2. En JavaScript, on utilise **removeEventListener()** pour supprimer un écouteur ajouté avec **addEventListener()**.

// 1. Définir une fonction séparée

```
function handleClick() {  
    console.log("Bouton cliqué !");  
}
```

// 2. Ajouter l'écouteur

```
btn.addEventListener("click", handleClick);
```

/ 3. Supprimer l'écouteur lorsqu'on clique sur "remove"

```
remove.addEventListener("click", () => {  
    btn.removeEventListener("click", handleClick);  
    console.log("Événement supprimé !");  
});
```

Manipulation du DOM : Web APIs (SetTimeout , setInterval)

- **setTimeout** est une Web API
 - Elle permet d'exécuter une fonction une seule fois, après un certain délai.

setTimeout(fonction, délai_en_millisecondes);

- **fonction** = le code à exécuter
- **délai_en_millisecondes**=temps d'attente (en ms)

```
setTimeout(( ) => {  
  
    console.log("Bonjour après 2 secondes");  
}, 2000);
```

=> Afficher Bonjour après 2 ms

Manipulation du DOM : Web APIs (SetTimeout , setInterval)

- **setInterval** est une Web API

Elle permet d'exécuter une fonction de manière répétée, avec un intervalle de temps fixe.

setInterval(fonction, délai_en_milliseconde);

- fonction = le code à exécuter
- délai_en_milliseconde=temps entre deux exécutions (en ms)

```
setInterval(() => {  
  
    console.log("Bonjour");  
  
}, 1000);
```

=> Afficher Bonjour chaque 1000 ms

Manipulation du DOM : Web Storage API

Stockage côté client avec JavaScript

- JavaScript peut stocker des données côté client.

Objectif : garder des informations entre les pages ou pendant une session.

- Les Web Storage API sont fournies par le navigateur

Web storage API	Durée de vie	Exemple d'usage
localStorage	Persistant (même après fermeture du navigateur)	Stocker les préférences de l'utilisateur
sessionStorage	Session en cours (fermeture onglet → effacé)	Stocker l'utilisateur connecté temporairement
Cookies	Définie par l'expiration	Gestion de session serveur, suivi analytics

Manipulation du DOM : Web Storage API

API sessionStorage : Stockage temporaire côté client, spécifique à un onglet.

Syntaxe :

```
sessionStorage.setItem("key", "value"); // Stocke une donnée
const value = sessionStorage.getItem("key"); // Récupère la donnée
sessionStorage.removeItem("key"); // Supprime une donnée
sessionStorage.clear(); // Vide tout
```

Utilité : transférer des données entre pages pendant une session.

Exemple:

```
// Stockage utilisateur après login
const user = { name: "Ahmed", balance: 1200 };
sessionStorage.setItem("currentUser", JSON.stringify(user));

// Récupération sur le dashboard
const currentUser = JSON.parse(sessionStorage.getItem("currentUser"));
console.log(currentUser.name); // "Ahmed"
```

Manipulation du DOM : Web Storage API

API localStorage: Stockage permanent côté client.

Syntaxe :

```
localStorage.setItem("key", "value"); // Stocke une donnée  
const value = localStorage.getItem("key"); // Récupère la donnée  
localStorage.removeItem("key")         // Supprime une donnée  
localStorage.clear();                  // Vide tout
```

Utilité : préférences, thèmes, tokens persistants.

Exemple:

```
localStorage.setItem("theme", "dark");  
const theme = localStorage.getItem("theme"); // "dark"
```

Manipulation du DOM : Web Storage API

Les Cookies: Stockage côté client pour sessions et suivi. Petit fichier texte stocké dans le navigateur.

Peut contenir :

- Identifiant de session
- Préférences utilisateur
- Données de suivi (analytics)

Envoyé automatiquement au serveur à chaque requête HTTP.

Syntaxe :

```
// Création d'un cookie
document.cookie = "username=Ahmed; max-age=3600; path="/";

// Suppression du cookie
document.cookie = "username=; max-age=0; path="/";
```

Utilité :

- Suivi des visites, pages consultées, interactions avec le site.
- Personnaliser les annonces en fonction des intérêts et du comportement de navigation.
- Stocker des choix de l'utilisateur : thème, langue, taille du texte, etc.

Les bases du JavaScript : JS object vs Json Object

Un objet JavaScript est une structure de données qui suit la notation key: value, où les clés ne sont pas nécessairement des chaînes et les valeurs peuvent être de tout type (fonctions, objets imbriqués, undefined, etc.).

```
const user = {  
  name: "Alice",  
  age: 25,  
  isAdmin: false,  
  greet: function() {  
    console.log("Hello, " + this.name);  
  }  
};  
  
console.log(user.name); // Alice  
user.greet(); // Hello, Alice
```

```
{  
  "name": "Alice",  
  "age": 25,  
  "isAdmin": false  
}
```

Objet JSON

Un objet JSON (JavaScript Object Notation) est une chaîne de texte qui suit un format spécifique, utilisé pour échanger des données entre serveurs et clients.

Les bases du JavaScript : JS object vs Json

Conversion entre Objet JavaScript et JSON

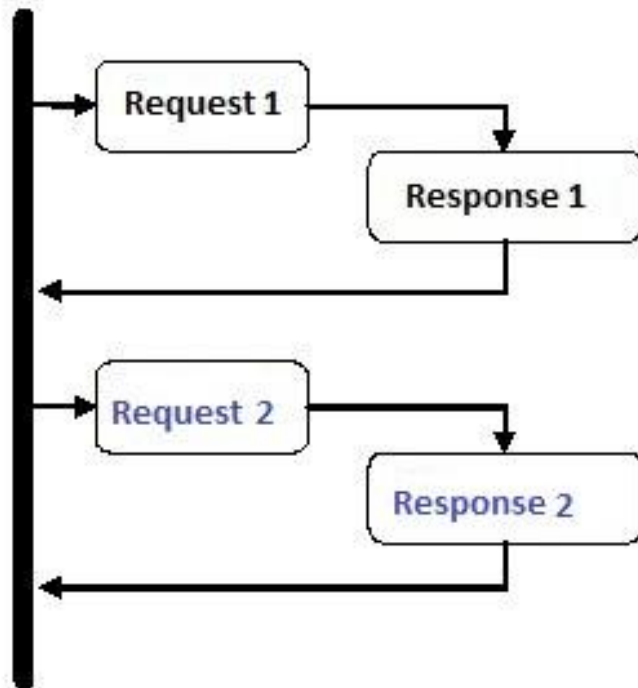
```
const user = {  
  name: "Alice",  
  age: 25,  
  isAdmin: false  
};  
  
const jsonString = JSON.stringify(user);  
console.log(jsonString);  
//  
{"name":"Alice","age":25,"isAdmin":false}
```

```
const jsonString = '{"name": "Alice",  
"age": 25, "isAdmin": false}';  
const userObject =  
JSON.parse(jsonString);  
  
console.log(userObject.name); //  
Alice  
console.log(userObject.age); // 25
```

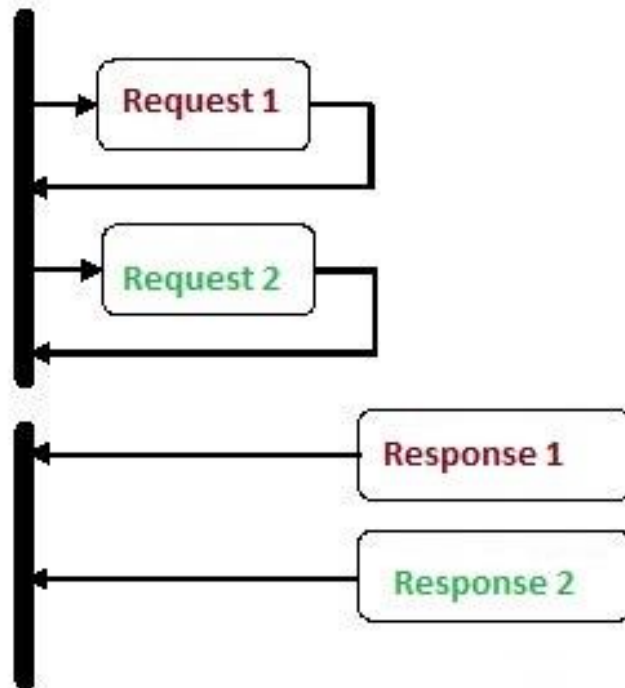
- Un **objet JavaScript** est utilisé en mémoire pour manipuler des données.
- Un **objet JSON** est une **chaîne** au format texte, utilisée pour le stockage et le transfert de données.

Synchrone vs Asynchrone code

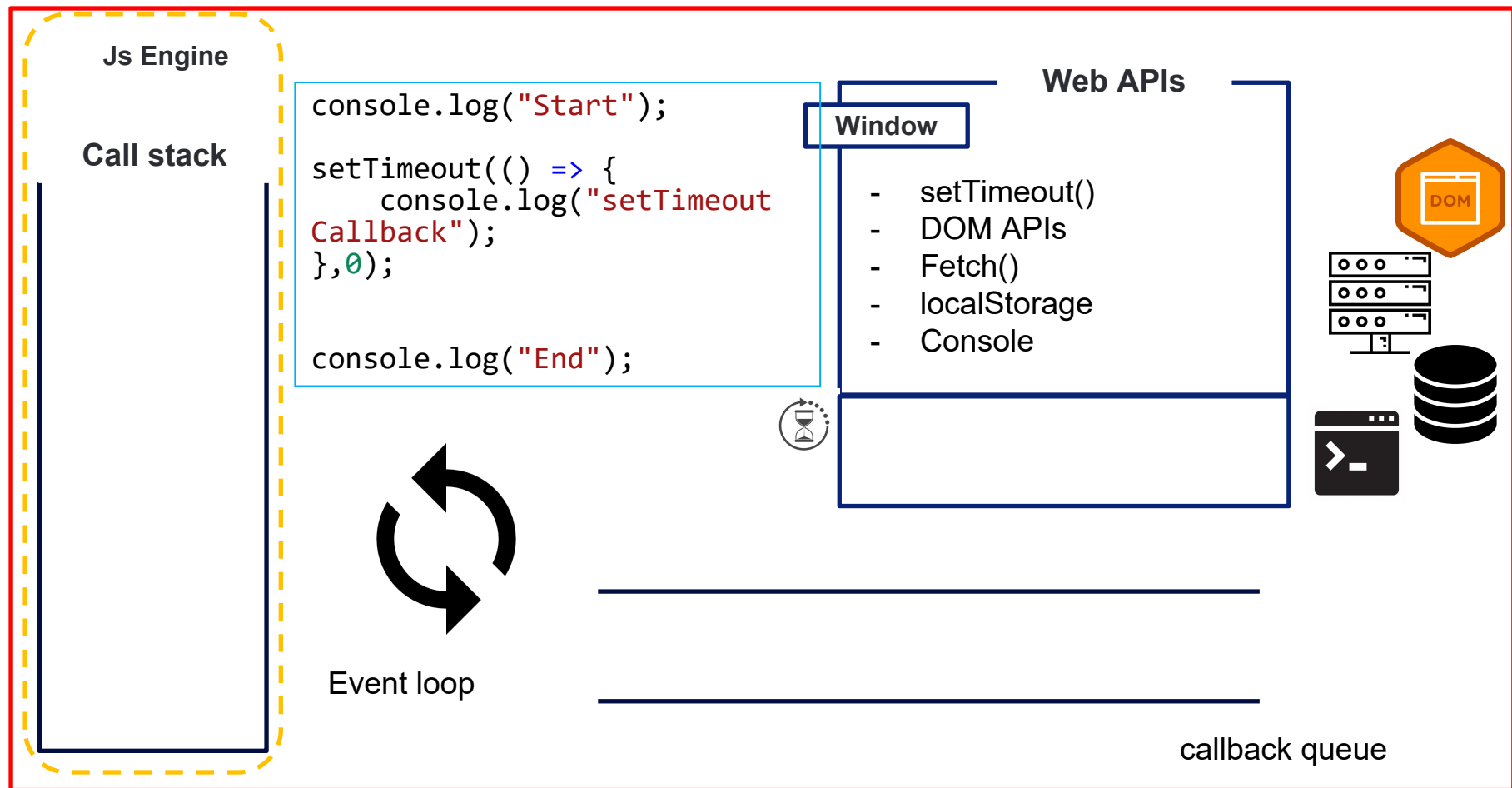
Synchronous



Asynchronous

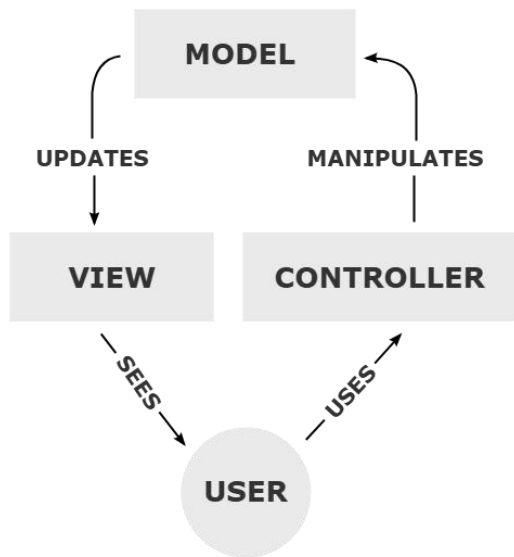


Les bases du JavaScript : Event loop



Atelier (Manipulation DOM et Asynchrone) : MVC Pattern

MVC est un pattern architectural qui organise le code en **trois** parties distinctes



Model (Modèle)

- Représente les données et la logique métier.
- Exemple : les utilisateurs, les produits,...etc.

View (Vue)

- Représente l'affichage à l'utilisateur.
- Exemple : HTML + CSS pour le login et le dashboard.

Controller (Contrôleur)

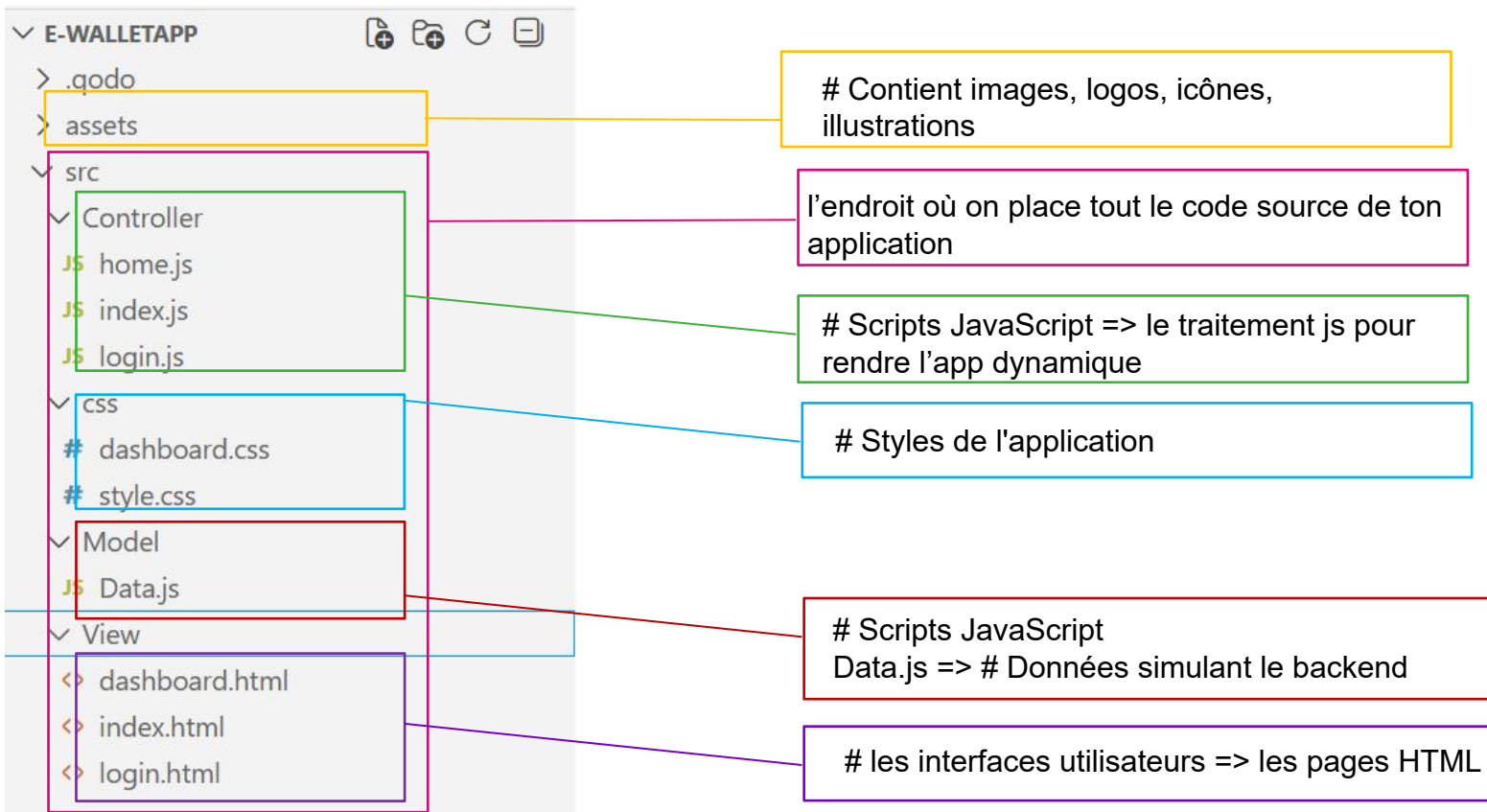
- Gère les interactions utilisateur et fait le lien entre Model et View.
- Exemple : gérer le clic sur le bouton login, vérifier l'utilisateur et mettre à jour l'affichage..

Avantage :

- **Séparation des responsabilités** : chaque partie a un rôle clair .
- **Réutilisable** : on peut appliquer MVC à n'importe quelle application web ou desktop.
- **Facilite le débogage et l'évolution** : un changement dans la View n'impacte pas le Model.
- **Prépare aux frameworks modernes** : React, Angular, Vue, Django utilisent des concepts similaires.

Atelier (Manipulation DOM et Asynchrone) : MVC Pattern

Etape1 : créer une arborescence claire, afin de séparer les rôles (HTML, CSS, JS, données).



Les bases du JavaScript : Notions Asynchrones

Le problème de gestion des callbacks imbriqués

```
getUser(userId, (user) => {  
  getOrders(user, (orders) => {  
    processOrders(orders, (processed) => {  
      sendEmail(processed, (confirmation) => {  
        console.log("Order Processed:", confirmation);  
      });  
    });  
  });  
});
```

- Lorsqu'on utilise des callbacks pour gérer des opérations asynchrones (Ex: setTimeout, des requêtes réseau ou des accès DB), on peut rencontrer ce qu'on appelle le « callback hell » ou « pyramid of doom ».
- Ce problème apparaît lorsque plusieurs callbacks sont imbriqués les uns dans les autres, rendant le code difficile à lire, à maintenir et à déboguer.
- Il survient principalement dans des scénarios où chaque opération asynchrone dépend du résultat de la précédente, ce qui entraîne une structure de code en forme de pyramide.

- **Lisibilité réduite** : Le code devient difficile à lire et à comprendre avec l'augmentation des callbacks imbriqués.
- **Gestion des erreurs complexe** : Chaque callback doit vérifier s'il y a une erreur, ce qui peut être lourd, répétitif et difficile à suivre.
- **Maintenance compliquée** : Ajouter de nouvelles étapes ou modifier des étapes existantes dans une chaîne de callbacks peut entraîner des erreurs ou des incohérences.

Le callback hell n'est pas un bug, c'est une mauvaise structure de code.

Les bases du JavaScript : Notions Asynchrones

Le problème L'Inversion de Contrôle dans les Callbacks

L'inversion de contrôle, dans le contexte des callbacks, signifie que le développeur ne maîtrise plus directement le moment d'exécution de son code.

En passant une fonction callback, l'exécution est confiée à l'Event Loop, ce qui peut rendre le flux du programme plus difficile à comprendre, surtout lorsque les callbacks deviennent imbriqués.

```
console.log("A"); // exécuté maintenant  
console.log("B"); // juste après  
console.log("C"); // juste après
```

```
console.log("A");  
✓ setTimeout(() => {  
  |   console.log("B");  
  }, 1000);  
  
console.log("C");
```

- Dans le code synchrone, le “quand” est immédiat et prévisible.
- Dans le code asynchrone, le “quand” est délégué au moteur JavaScript.

Les bases du JavaScript : Notions Asynchrones

Le problème L'Inversion de Contrôle dans les Callbacks

L'inversion de contrôle, dans le contexte des callbacks, fait référence au fait que vous perdez une partie du contrôle sur l'exécution de votre code lorsque vous passez une fonction **callback** à une autre fonction. La fonction d'origine (celle qui appelle le callback) ne sait pas exactement quand la fonction de rappel sera exécutée, ni dans quel ordre. Cela crée une forme de **délégation de contrôle** qui peut entraîner une complexité croissante à mesure que le code devient plus imbriqué.

```
// Fonction principale
function createCommande() {
  console.log('Début de la commande...');

  getDataFromServer(function(data) {
    processData(data);
    saveData(data);
  });

  console.log('Commande en cours...');
}

createCommande();
```

Le problème principal ici est que la fonction **createCommande** délègue le contrôle à la fonction de rappel dans **getDataFromServer**, et elle ne peut pas contrôler l'ordre d'exécution des actions suivantes. Cela devient problématique quand :

- Il y a beaucoup de callbacks imbriqués.
- Le flux du programme devient difficile à suivre.
- Vous devez faire des appels multiples dans un ordre précis, et les erreurs dans l'une des étapes peuvent entraîner des effets en chaîne.

Les bases du JavaScript : Notions Asynchrones (promise)

Une **Promise** en JavaScript est un objet représentant une **opération asynchrone** (qui prend du temps) qui **n'est pas encore terminée**, mais qui finira bien par donner un résultat dans le futur, ou échouer.

```
// creation de la promesse
let verifierSiPair =(nombre) =>{
  return new Promise((resolve, reject) => {
    if (nombre % 2 === 0) {
      resolve(`Le nombre ${nombre} est pair.`);
    } else {
      reject(`Le nombre ${nombre} est impair.`);
    }
  });
}

// Utilisation de la promesse
verifierSiPair(8)
  .then(message => console.log(message)) // succès (résolu)
  .catch(erreur => console.log(erreur)); //échec (rejeté)
```

Les promesses permettent d'enchaîner plusieurs opérations asynchrones de manière plus lisible et gérable, tout en centralisant la gestion des erreurs.

Une **Promise** peut être dans trois états différents :

1. **En attente (Pending)** : La promesse n'est pas encore terminée. L'opération asynchrone est en cours.
2. **Réussie (Fulfilled)** : L'opération a été terminée avec succès, et tu as le résultat que tu attendais.
3. **Échouée (Rejected)** : L'opération a échoué, et il y a une erreur ou un problème.*

Les bases du JavaScript : Notions Asynchrones (Promise)

Les promesses permettent de mieux gérer les opérations asynchrones en offrant une syntaxe plus claire et en réduisant les risques de "callback hell".

```
function fetchData(url) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = true; // Simulons un succès
      if (success) {
        resolve("Données de l'API");
      } else {
        reject("Erreur lors de la récupération des données");
      }
    }, 2000);
  });
}

fetchData("https://exemple.com")
  .then((data) => {
    console.log("Réponse : ", data);
  })
  .catch((error) => {
    console.log("Erreur : ", error);
  });

return fetchData("https://exemple2.com"); // Retourne une nouvelle promesse
```

- **Chaining (Chaînage)** : Les promesses permettent d'enchaîner plusieurs opérations asynchrones de manière plus lisible, sans avoir à imbriquer les fonctions callback.
- **Gestion des erreurs améliorée** : Avec les promesses, les erreurs sont capturées de manière centralisée via `.catch()`, ce qui rend la gestion des erreurs plus simple et plus lisible.
- **Syntaxe plus claire** : Moins de risque de "callback hell" avec le chaînage des promesses. Cela rend votre code plus lisible et plus facile à maintenir.
- **Gestion asynchrone plus fluide** : Une promesse peut être en attente (**pending**), résolue (**fulfilled**) ou rejetée (**rejected**), ce qui permet de mieux gérer les flux d'exécution.

Les bases du JavaScript : Notions Asynchrones (Promise)

méthodes `.then()` et `.catch()` :

- **`.then()`** : Cette méthode est utilisée pour gérer la résolution d'une Promise. Elle prend une fonction callback qui sera exécutée lorsque la Promise sera résolue avec succès (c'est-à-dire, lorsque l'état de la Promise passe de en attente à résolue). La valeur que la Promise renvoie à l'intérieur de la fonction `resolve()` sera passée en paramètre à la fonction de rappel dans `.then()`.
- **`.catch()`** : Cette méthode est utilisée pour gérer le rejet d'une Promise. Elle prend une fonction callback qui sera exécutée lorsque la Promise sera rejetée. La raison du rejet (en général un objet d'erreur) sera passée en paramètre à la fonction de callback dans `.catch()`.

```
let checkEven = new Promise((resolve, reject) => {  
  let number = 4;  
  if (number % 2 === 0)  
    resolve("The number is even!");  
  else  
    reject("The number is odd!");  
});  
checkEven  
  .then((message) => console.log(message)) // On success  
  .catch((error) => console.error(error)); // On failure
```

Les bases du JavaScript : Notions Asynchrones (async /await)

async / await est une syntaxe moderne de JavaScript qui permet d'écrire du code asynchrone de manière plus lisible, plus proche du code synchrone.

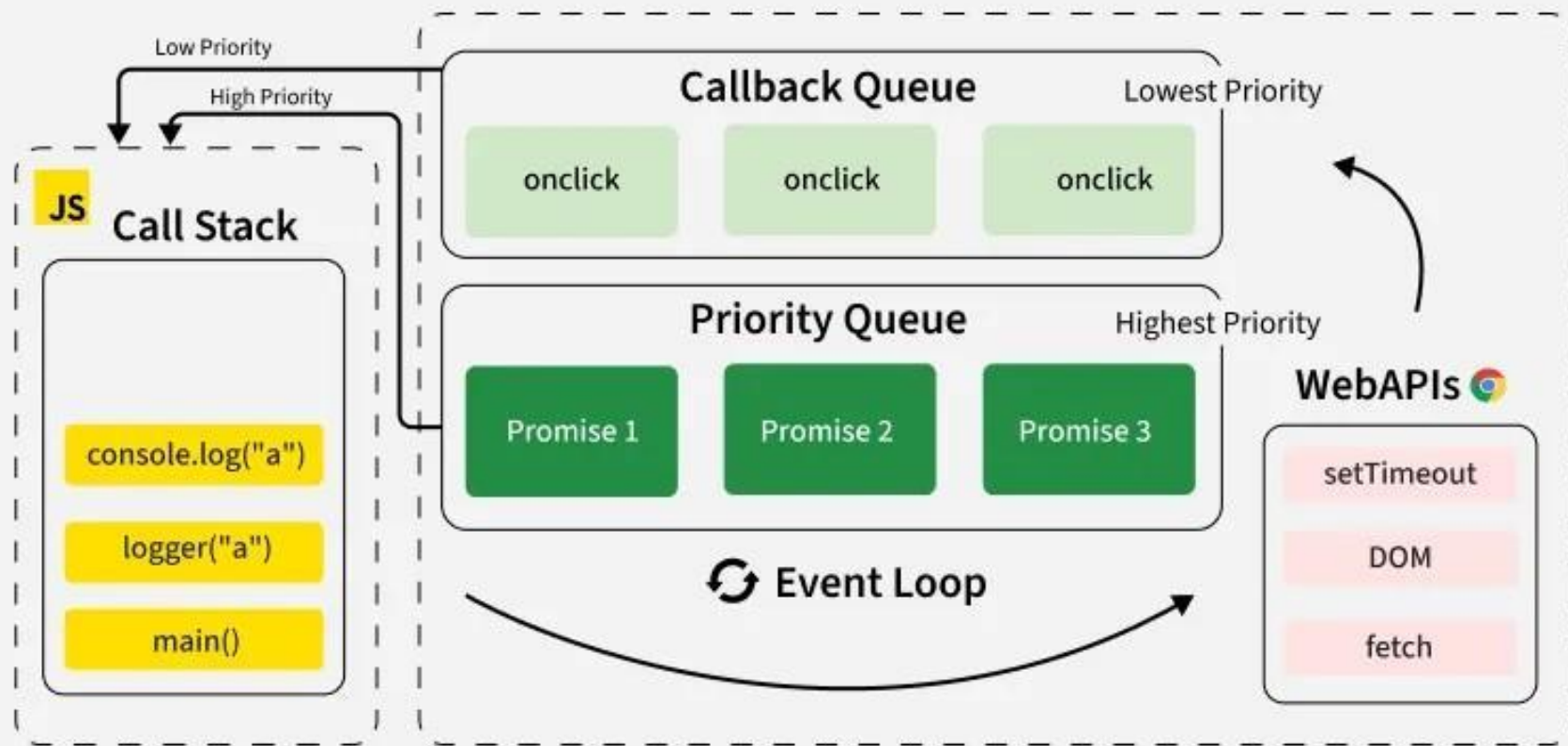
- **async / await** repose entièrement sur les Promises
- Ce n'est pas un nouveau mécanisme, seulement une nouvelle écriture
- **async / await** ne change PAS la création de la Promise=> Il change uniquement la façon de l'utiliser.

```
let checkEven = new Promise((resolve, reject) => {  
  let number = 4;  
  if (number % 2 === 0)  
    resolve("The number is even!");  
  else  
    reject("The number is odd!");  
});
```

```
checkEven  
  .then((message) => console.log(message))//succes  
  .catch((error) => console.error(error));//echec
```

```
async function testerNombre() {  
  const message = await checkEven;  
  console.log(message); // succès  
}  
testerNombre();
```

Les bases du JavaScript : promises



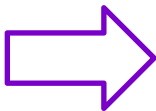
Les bases du JavaScript : Responsabilité Unique (SRP)

La Solution = l'utilisation des **services** pour implementer le principe de Responsabilité Unique (SRP).

Service : Un service est une unité logique indépendante du UI. Il permet de centraliser les actions techniques (API, Storage, logique métier) pour rendre le code réutilisable et plus facile à tester.

En utilisant des services, nous transformons une structure désordonnée en une architecture professionnelle inspirée de frameworks comme NestJS

```
src > Controller > JS dashboard.js > ...
1  let user = JSON.parse(sessionStorage.getItem("user"));
2  const welcome_message = document.getElementById("welcome_message");
3  const balance = document.getElementById("balance");
4  const transactionsBody = document.querySelector("#transactions tbody");
5  const filterType = document.getElementById("filterType");
6  const transfertbtn=document.getElementById("transferrer");
7  welcome_message.textContent = `Bienvenue ${user.name}`;
8  balance.textContent = `${user.balance} MAD`;
9  transfertbtn.addEventListener("click",handletransfer);
10 | //AFFICHAGE TRANSACTIONS
   Qodo: Test this function
11 > function afficherTransactions(list) { ...
42 }
43 | //FILTRAGE
44
45 > filterType.addEventListener("change", () => { ...
56 });
57 | // AFFICHAGE INITIAL
58 afficherTransactions(user.transactions);|
   Qodo: Test this function
59 function handletransfer(){
60     setTimeout(()=>{
61         window.location.href="/src/View/Transfer.html"
62     },1000);
```



-> Gestion UI

-> la logique métier

Les bases du JavaScript : Gestion des erreurs try/catch

Pourquoi la gestion des erreurs est essentielle ?

Dans une application réelle :

- une requête peut échouer
- une donnée peut être invalide
- un service externe peut être indisponible
- une règle métier peut être violée

Sans gestion des erreurs :

- l'application plante
- l'utilisateur ne comprend pas ce qui se passe
- le système devient difficile à maintenir

La gestion des erreurs permet de rendre l'application fiable, lisible et robuste.

Les bases du JavaScript : Gestion des erreurs try/catch

Permettre au programme de continuer à s'exécuter même lorsqu'une erreur survient en interceptant et en gérant l'exception

Bloc try

- Contient le code susceptible de générer une erreur
- Si une erreur se produit, l'exécution passe au bloc catch correspondant.

Bloc catch

- Intercepte et gère l'exception
- Permet d'exécuter du code alternatif en cas d'erreur

Bloc finally (Optionnel)

- Toujours exécuté, que l'erreur survienne ou non
- Idéal pour nettoyer des ressources (fermer fichiers, libérer mémoire ...)

Flux d'exécution

- Le code du bloc try est exécuté
- Si pas d'erreur → catch ignoré
- Si erreur → exécution de catch
- finally s'exécute toujours (si présent)

Les bases du JavaScript : Gestion des erreurs try/catch

Syntaxe générale:

```
try {  
    // code à risque  
} catch (error) {  
    // gestion de l'erreur  
} finally {  
    // code exécuté dans tous les cas  
}
```

- On ne peut pas utiliser catch sans try
- On ne peut pas utiliser plusieurs try avec un seul catch partagé
- On peut avoir plusieurs catch pour gérer différents types d'erreurs
- Le finally est facultatif mais très utile pour la propreté du code

La structure est similaire dans de nombreux langages (C++,JS,Java, Python).

Les bases du JavaScript : throw + try / catch

Syntaxe générale:

```
async function payer(amount) {  
  if (amount <= 0) {  
    thrownew Error("Montant invalide");  
  }  
  return"Palement réussi";  
}  
try {  
  await payer(-10);  
} catch (e) {  
  console.log(e.message);  
}
```

throw permet de lever (déclencher) une erreur volontairement.

On utilise **throw** lorsqu'une règle métier n'est pas respectée ou lorsqu'une situation est invalide, même si JavaScript ne génère pas automatiquement d'erreur.

throw permet de signaler explicitement une erreur et de forcer son traitement via **try / catch**.

Js coté serveur : Framework Nest JS

Pourquoi un backend ?

- Notre application E-wallet en JavaScript natif fonctionne... mais seulement en apparence

Problèmes du front-end seul

- Les données sont stockées dans le navigateur (sessionStorage, localStorage)
- Les données disparaissent si on change de navigateur ou d'appareil
- N'importe qui peut modifier le code côté client
- Aucune vraie sécurité
- Aucune gestion multi-utilisateurs réelle

Un front-end seul n'est pas fiable pour une application réelle.

Js coté serveur : Framework Nest JS

Le backend est le cerveau et la mémoire de l'application.

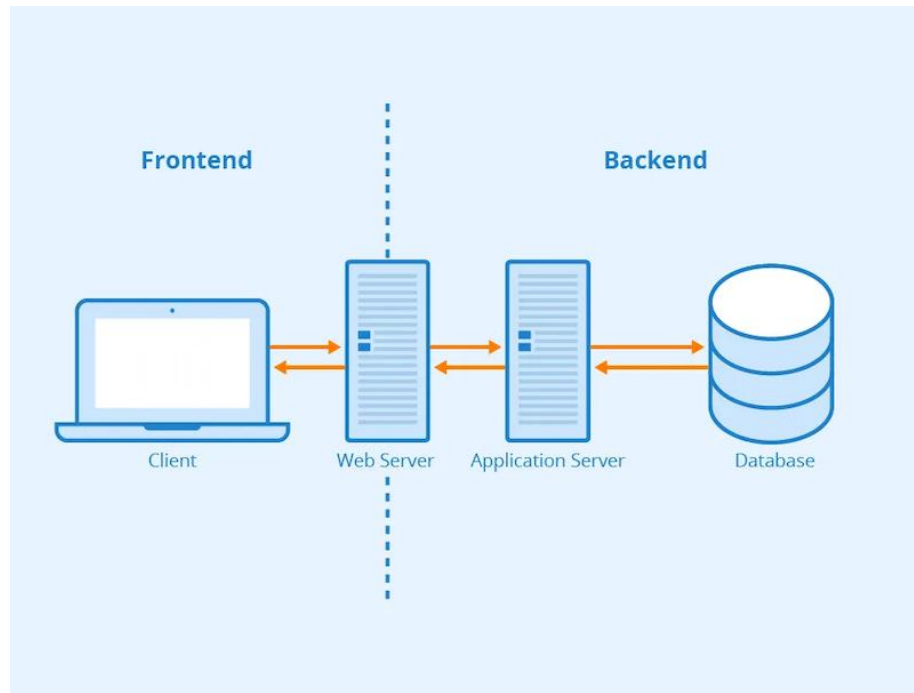
Il permet de :

- stocker les données de manière permanente
- appliquer les règles métier
- sécuriser les opérations
- gérer plusieurs utilisateurs simultanément

Le **backend** désigne tout ce qui fonctionne côté serveur.

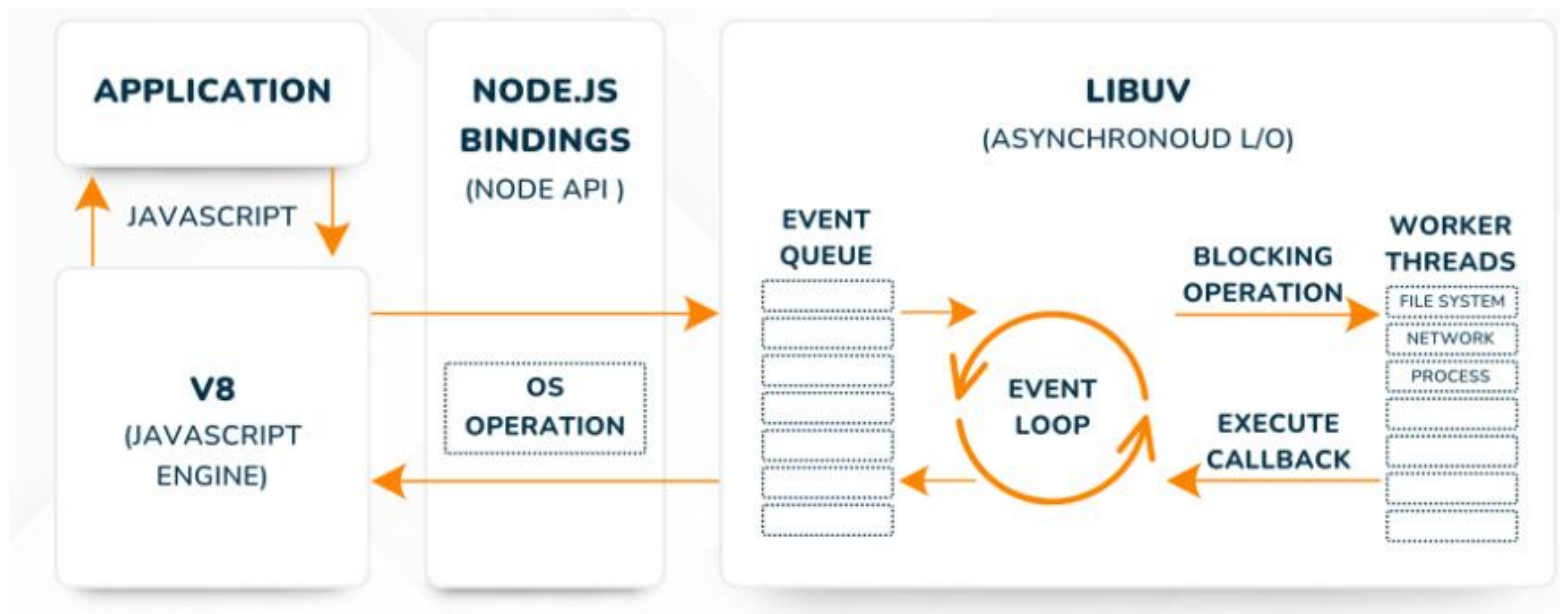
Il inclut généralement :

- un web server (gestion HTTP)
- un application server (logique métier)
- une base de données
- parfois des services externes (auth, paiement...)



JavaScript côté serveur : Environnement Node.js

Node.js est un environnement d'exécution écrit en C++ qui permet d'exécuter du JavaScript côté serveur.



JavaScript côté serveur : Environnement Node.js

installer l'environnement d'exécution Node.js

Étape 1 : Télécharger Node.js (LTS)

- Accéder au site officiel de Node.js
- Télécharger la version LTS (Long Term Support)
- Lancer le programme d'installation

Étape 2 : Installer Node.js

- Pendant l'installation :
- Conserver les options par défaut
- Vérifier que l'option "Add to PATH" est activée
- Laisser l'option "Automatically install the necessary tools..." désactivée (facultatif)

Étape 3 : Vérifier l'installation

- Ouvrir un terminal (CMD, PowerShell ou Terminal)
- Vérifier les versions installées :

```
node -v  
npm -v
```

npm (Node Package Manager)

npm est le gestionnaire de paquets officiel de Node.js.

Il permet de :

- Installer des bibliothèques
- Gérer les dépendances d'un projet
- Exécuter des scripts

JavaScript côté serveur : Environnement Node.js

Etape 4: Installation de NestJS

4.1 - Créer le dossier du projet

- Créer un dossier nommé **Back-end**
- Ce dossier contiendra tout le code côté serveur

4.2 - Ouvrir le dossier dans un éditeur de code

- Ouvrir le dossier Back-end avec un éditeur de code (exemple : Visual Studio Code)

4.3 - Ouvrir le terminal

- Ouvrir le terminal intégré de l'éditeur (Terminal → New Terminal)
- Vérifier que le terminal est bien positionné dans le dossier Back-end

4.4 - Installation et utilisation de la CLI NestJS

- **npx @nestjs/cli new Ewallet_Backend**
- Cette commande exécute la CLI NestJS sans installation globale.
- Elle permet de générer automatiquement l'architecture initiale du projet.

4.5- Démarrer le serveur web NestJS

- **npm run start :dev**
- Lance le serveur en mode dev (Redémarrage automatique lors des modifications du code)

Introduction à NestJS



Nest JS



express

NestJS est un framework backend basé sur Node.js. Il utilise TypeScript et repose sur Express pour gérer les requêtes HTTP. Il propose une architecture **modulaire**, l'**injection de dépendances** et des **décorateurs** pour organiser le code. NestJS est un framework opinioné.

Les Modules

- Un module regroupe les parties de l'application liées à une même fonctionnalité.
- Toute application NestJS contient au moins un module : le module racine (AppModule)

Les Contrôleurs

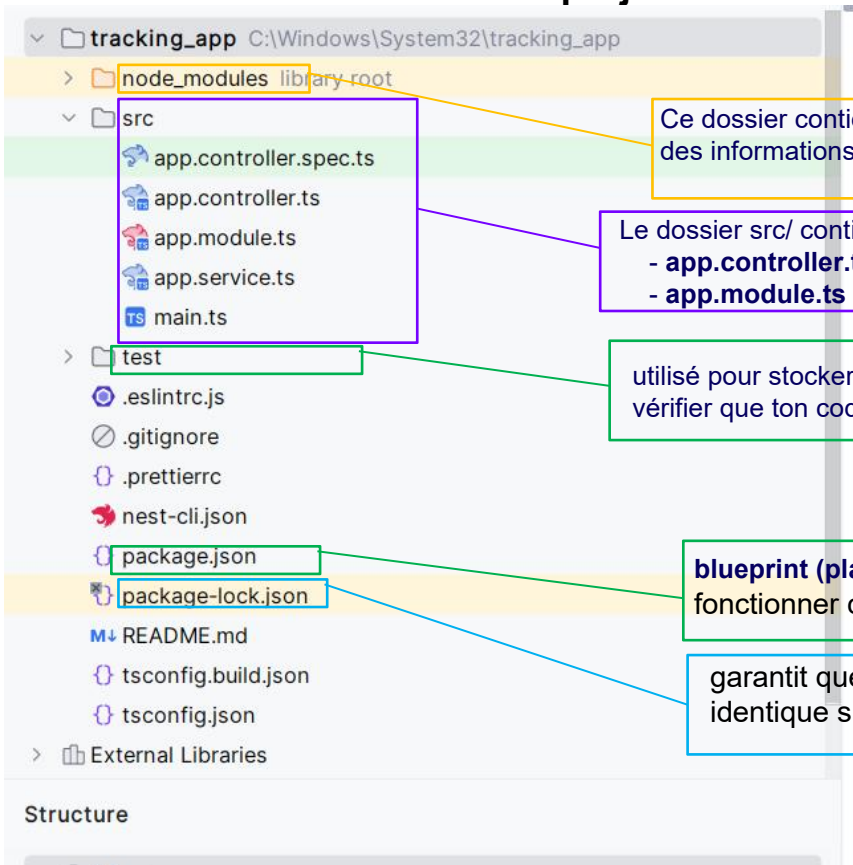
- Les contrôleurs reçoivent les requêtes HTTP et renvoient les réponses.
- Ils définissent les routes avec des décorateurs comme `@Get()`, `@Post()`, etc.

Les Services (Providers)

- Les services contiennent la logique métier.
- Ils sont utilisés par les contrôleurs grâce à l'injection de dépendances.

Back End Server Nestjs

D- Vérifier la structure du projet :



Ce dossier contient toutes les dépendances installées par npm en fonction des informations de package.json.

Le dossier src/ contient le code source de l'application.
- **app.controller.ts** contient le contrôleur principal de votre application.
- **app.module.ts** est où vous configurez vos modules

utilisé pour stocker les fichiers de tests. Ces tests permettent de vérifier que ton code fonctionne correctement.

blueprint (plan) indique à **npm** ce dont ton projet a besoin pour fonctionner correctement.

garantit que les dépendances de ton projet seront installées de manière identique sur toutes les machines (ton ordinateur, ou sur un serveur).

La structure de base d'une application NestJS

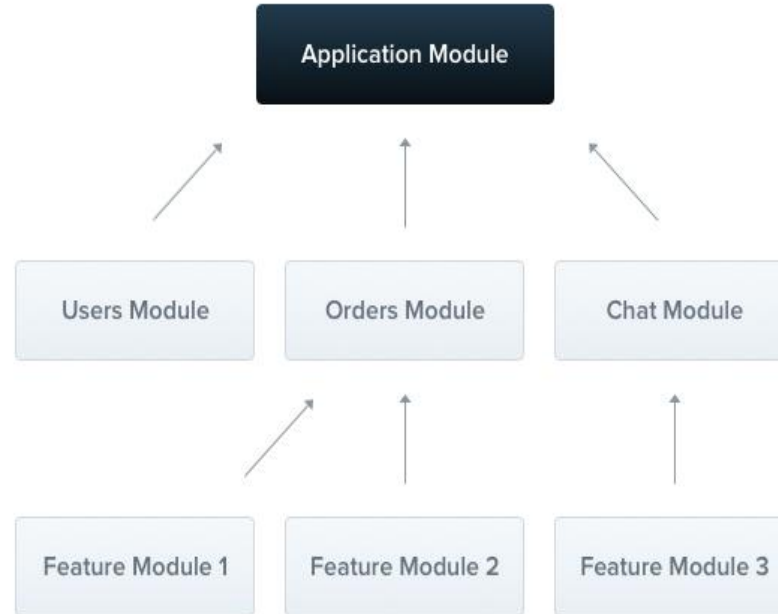


Fichier	Description	Exemple d'utilisation
app.controller.ts	Contrôleur de base avec une seule route . Il gère les requêtes HTTP entrantes et retourne les réponses appropriées. - Utilise des décorateurs tels que @Controller et @Get (ainsi que d'autres méthodes HTTP comme @Post, @Put, etc.) pour définir une route	<pre>import { Controller, Get } from '@nestjs/common'; @Controller('example') // Prefix for the route: /example export class AppController { @Get() // Handles GET requests to /example getHello(): string { return 'Hello, World!'; } }</pre>
app.module.ts	Module racine de l'application. Organise les contrôleurs, les services et importe d'autres modules. Un module est un regroupement logique de composants responsables d'une fonctionnalité ou d'une fonctionnalité spécifique de l'application.	<pre>... @Module({ // Show usages imports: [AuthentModule, UserModule], controllers: [AppController, AuthController, UserController], providers: [AppService, UserService, AuthService], }) export class AppModule {} ...</pre>
app.service.ts	Service de base avec une méthode simple. Contient la logique métier et est utilisé par les contrôleurs pour exécuter des tâches.	<pre>1 import { Injectable } from '@nestjs/common'; 2 ... 3 @Injectable() // Show usages 4 export class AppService { 5 getHello(): string { // Show usages 6 const text = 'Hello ILISI3'; 7 return text; 8 } }</pre>
main.ts	Point d'entrée de l'application. Utilise NestFactory pour créer une instance de l'application NestJS et définir les configurations globales.	<pre>async function bootstrap(): Promise<void> { // Show usages const app : INestApplication<any> = await NestFactory.create(AppModule); await app.listen(process.env.PORT ?? 3000); } bootstrap();</pre>

La structure de base d'une application NestJS



Le module racine constitue le point d'entrée de l'application NestJS. Il permet à Nest de construire un graphe interne décrivant les relations entre les modules et les fournisseurs afin de gérer l'injection des dépendances.

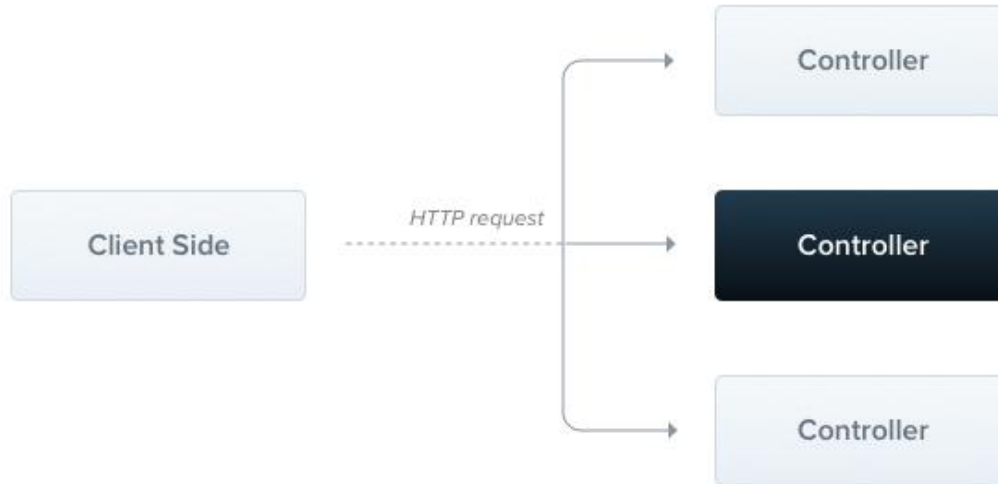


La structure de base d'une application NestJS



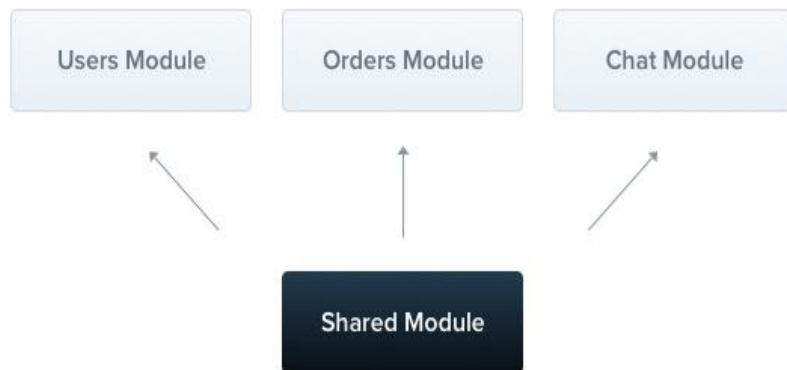
Les contrôleurs sont responsables du traitement des requêtes entrantes et de l'envoi des réponses au client.

Les contrôleurs reçoivent les requêtes HTTP, appellent les services appropriés et renvoient les réponses au client.



La structure de base d'une application NestJS

Les providers d'un module sont des singletons par défaut. Pour les partager entre modules, ils doivent être exportés..

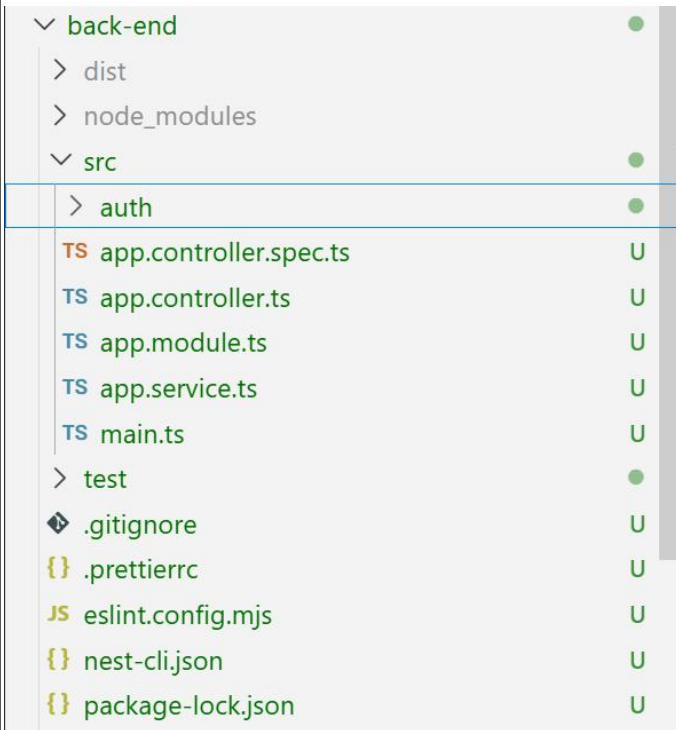


Chaque module est automatiquement un module partagé. Une fois créé, il peut être réutilisé par n'importe quel module.

Back End Server Nest

Étape 2 : Création des Modules et API (Module 1 : Authentification)

```
npx nest generate module auth  
npx nest generate controller auth  
npx nest generate service auth
```

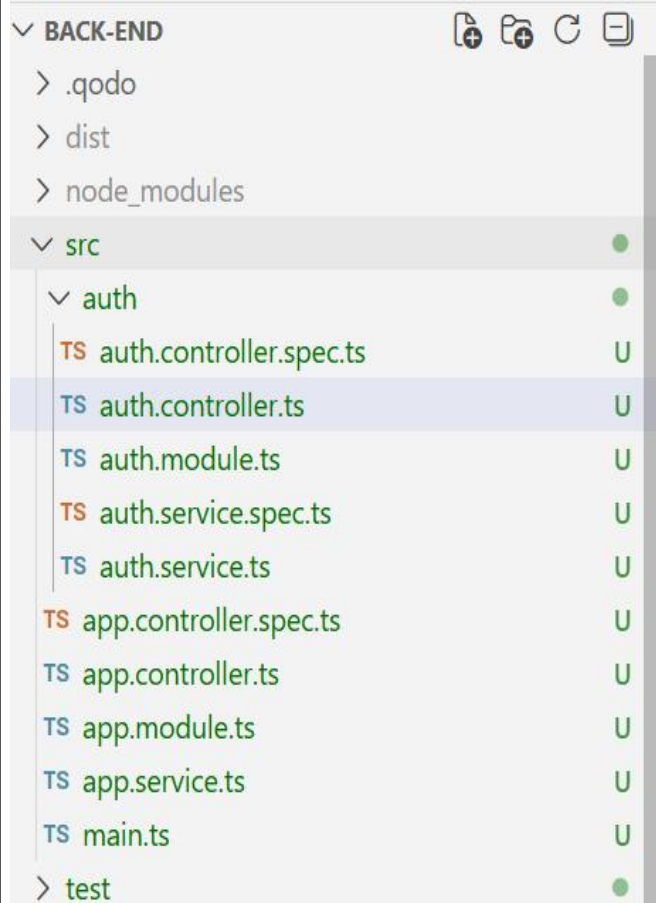


auth.module.ts

```
import { Module } from '@nestjs/common';  
import { authController } from './auth.controller';  
import { authService } from './auth.service';
```

```
@Module({  
  controllers: [authController],  
  providers: [authService],  
})  
export class authModule {}
```

Back End Server Nest



Auth.controller.ts

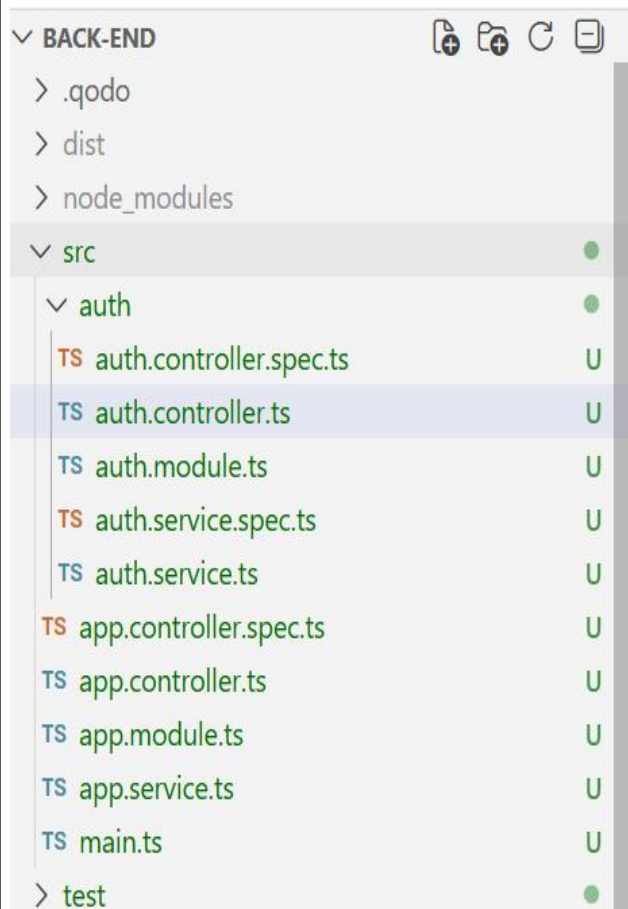
```
import { Body, Controller, Get, Post, Query } from "@nestjs/common";
import { AuthService } from "../auth.service";

@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @Post('register')
  register(@Body() body: {email: string, password:string}): string {
    const {email, password}= body;
    return this.authService.register(email, password);
  }

  @Post('login')
  login(@Body() body: {email: string, password: string}){
    console.log(body)
    const {email, password}= body;
    return this.authService.login(email, password);
  }
}
```

Back End Server Nest



Auth.service.ts

```
import { Injectable } from '@nestjs/common';
```

```
@Injectable()
```

```
export class AuthService {
```

```
  private users= []; // tableau pour stocker les utilisateur
```

```
  register(email: string , password : string) : string {
```

```
    this.users.push({email:email,password:password});
```

```
    return 'Utilisateur enregistré avec succès';
```

```
  }
```

```
  login(email: string , password : string) : string {
```

```
    const user = this.users.find(
```

```
      (user)=>user.email===email&& user.password===password);
```

```
    return user ? 'connexion reussie': 'cridentials invalide';
```

```
  }}
```


Back End Server Nest

fetch est une API JavaScript native permettant d'envoyer des requêtes HTTP (GET, POST, PUT, DELETE...) vers un serveur.

- Intégrée au navigateur
- Basée sur les Promises
- Utilisée pour communiquer avec une API (backend)

Syntaxe de base

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

Syntaxe avec async (recommandé)

```
async function getData() {
  const response = await fetch(url);
  const data = await response.json();
  console.log(data);
}
```

Exemple Fetch avec Post

```
fetch("url", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({login_data}),
});
```

Back End Server Nest

- **JSON.stringify** transforme un objet JS en JSON
- **Content-Type** indique le format envoyé au serveur
- **fetch** retourne une Promise
- Une fois **résolue**, cette Promise donne un objet **Response**
- **response** est un objet HTTP, pas les données directement.

Il contient par exemple :

- response.status → code HTTP (200, 404, 500...)
- response.ok → true / false
- response.headers → en-têtes HTTP
- response.json() → méthode pour lire le body

Exemple Fetch avec Post

```
const response = await fetch("http://localhost:3000/Ewallet/login", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify({ mail, password }),  
});  
console.log(response);
```

```
const response = await fetch(url);  
console.log(response.status); // 200  
console.log(response.ok);    // true  
const data = await response.json();  
console.log(data);
```

Les données envoyées par le serveur ne sont pas directement accessibles.

Back End Server Nest

JSON (JavaScript Object Notation) est un format de données textuel, léger et lisible, utilisé pour échanger et stocker des données structurées.

Caractéristiques

- Format texte
- Basé sur les objets JavaScript
- Facile à lire par l'humain
- Facile à parser par les machines

Utilisé dans :

- APIs REST
- Frontend ↔ Backend
- Fichiers de configuration
- Stockage simple de données

Structure d'un fichier JSON

```
{  
  "id": 1,  
  "email": "user@test.com",  
  "password": "1234",  
  "role": "USER",  
  "isActive": true  
}
```

Types de données supportés

- string
- number
- boolean
- object
- array
- null

Back End Server Nest

JSON (JavaScript Object Notation) est un format de données textuel, léger et lisible, utilisé pour échanger et stocker des données structurées.

Lire un fichier JSON

```
import { promises as fs } from 'fs';
import * as path from 'path';

async getUsers() {
  const filePath = path.join(__dirname, '..', 'data', 'users.json');

  const data = await fs.readFile(filePath, 'utf-8');
  return JSON.parse(data);
}
```

fs.readFile → lit le fichier

JSON.parse → transforme le JSON en objet JavaScript

path.join → chemin portable

import { promises as fs } from 'fs' : On importe la version asynchrone du module fs (File System).
import * as path from 'path'; Le module path permet de gérer les chemins de fichiers de façon portable.

process.cwd(): **Current Working Directory**

Back End Server Nest

JSON (JavaScript Object Notation) est un format de données textuel, léger et lisible, utilisé pour échanger et stocker des données structurées.

Ajouter des données (Ecriture)

```
async addUser(user) {  
  const filePath = path.join(__dirname, '..', 'data', 'users.json');  
  
  const data = await fs.readFile(filePath, 'utf-8');  
  const users = JSON.parse(data);  
  
  users.push(user);  
  
  await fs.writeFile(filePath, JSON.stringify(users, null, 2));  
}
```

fs.writeFile(...)

Écrit des données dans un fichier.

- Si le fichier existe → il est écrasé
- S'il n'existe pas → il est créé

Paramètre	Rôle
users	Objet JS à convertir
null	Aucun filtre
2	Indentation (lisibilité)

Installation de PostgreSQL

1. Télécharger PostgreSQL

<https://www.postgresql.org/download/>

Choisis le système d'exploitation (windows, linux, macOS)



2. Installation sous Windows

Étapes :

1. Lancer le fichier .exe
2. Cliquer sur Next
3. Choisir le dossier d'installation (laisser par défaut)
4. Sélectionner les composants (recommandé) :
 - PostgreSQL Server
 - pgAdmin 4
 - Command Line Tools

5. Définir le mot de passe du super-utilisateur postgres

- Note-le bien (important pour PostGIS, pgAdmin, TypeORM...)
- Port : 5432 (par défaut, ne pas changer)
- Locale : Default
- Install → Finish

NestJs : PostgreSQL avec TypeORM

Etape 1 : Installer les dépendances nécessaires

installer les paquets nécessaires pour TypeORM et PostgreSQL

```
npm install @nestjs/typeorm typeorm pg
```

Etape 2 : Configurer TypeORM :

configurer la connexion à la base de données et déclarer les entités.

```
6 import { TypeOrmModule } from "@nestjs/typeorm";
7 import { User } from './user/entities/user.entity';
  Qodo: Test this class
8 @Module({
9   imports: [
10    //configuration de TypeORM
11    TypeOrmModule.forRoot({
12      type: 'postgres',
13      host: 'localhost',
14      port: 5433,
15      username: 'postgres',
16      password: 'admin',
17      database: 'BusTracking_db',
18      entities: [User],
19      synchronize: true, // DEV uniquement
20    }),AuthModule, UserModule],
21    controllers: [AppController],
22    providers: [AppService],
23  })
```

TypeOrmModule.forRoot configure la connexion à la base de données et doit être importé une seule fois dans le module **racine** de l'application.

l'option **synchronize** met automatiquement à jour le schéma de la base de données à partir des entités. Elle est utile en phase de développement, mais dangereuse en environnement de production.

NestJs : TypeORM

Etape 3 : Créer une Entity

back-end > src > user > entities > TS user.entity.ts >  User >  fullName

```
1 import { Entity, PrimaryGeneratedColumn,
2         Column, CreateDateColumn, } from "typeorm";
3 Qodo: Test this class
4 @Entity("users")
5 export class User {
6     @PrimaryGeneratedColumn()
7     id: number;
8     @Column({ unique: true })
9     email: string;
10    @Column()
11    password: string;
12    @Column()
13    fullName: string;
14    @Column()
15    role: string;
16    @CreateDateColumn()
17    createdAt: Date;
18 }
```

Un module peut contenir plusieurs entités,
tant qu'elles appartiennent au même domaine
métier

Etape 4: Importer l'Entity dans son Module

Rendre l'entité exploitable via un Repository,
puis l'utiliser dans un service

back-end > src > user > TS user.module.ts > ...

```
1 import { Module } from '@nestjs/common';
2 import { UserController } from '../user.controller';
3 import { UserService } from '../user.service';
4 import { TypeOrmModule } from '@nestjs/typeorm';
5 import { User } from '../entities/user.entity';
6
7 Qodo: Test this class
8 @Module({
9     imports: [TypeOrmModule.forFeature([User])],
10    controllers: [UserController],
11    providers: [UserService]
12 })
13 export class UserModule {}
```

forFeature([User]) : Active le repository
(Repository<User>) d'une entité dans un module

NestJs : TypeORM

Etape 5: Injection du Repository

Injecter le repository dans un service et implémenter les premières méthodes CRUD

back-end > src > user > TS user.service.ts > UsersService > constructor

```
1 import { Injectable } from '@nestjs/common';
2 import { Repository } from "typeorm";
3 import { InjectRepository } from "@nestjs/typeorm";
4 import { User } from "../entities/user.entity";
5
6 Qodo: Test this class
7 @Injectable()
8 export class UsersService {
9   constructor(
10     @InjectRepository(User)
11     private readonly userRepository: Repository<User>
12   ) {}
13 }
```

- **@InjectRepository(User)** : demande à NestJS le repository activé

- **Repository<User>** est l'API CRUD fournie par TypeORM

- Le service est le seul endroit qui parle à la DB

Étape 6 — Écrire les méthodes CRUD de base

- create() → crée l'objet (mémoire)
- save() → INSERT/UPDATE
- findOne() → SELECT
- find() → SELECT *

```
create(user: Partial<User>) {
  const entity = this.userRepository.create(user);
  return this.userRepository.save(entity);
}

// Trouver un utilisateur par email
Qodo: Test this method
findByEmail(email: string) {
  return this.userRepository.findOne({ where: { email } });
}
```

NestJs : Les relations TypeORM

Objectif

Relier des objets entre eux comme dans le monde réel.

Exemples réels

- Un User possède plusieurs Accounts
- Un Account contient plusieurs Transactions
- Une Transaction appartient à un seul Account

Les relations permettent de modéliser ces liens

Relation	Signification
OneToOne	$1 \leftrightarrow 1$
OneToMany	$1 \rightarrow N$
ManyToOne	$N \rightarrow 1$
ManyToMany	$N \leftrightarrow N$

NestJs : Les relations TypeORM

Relation OneToMany / ManyToOne (la plus utilisée)

Exemple métier

- Un User peut avoir plusieurs Accounts
- Un Account appartient à un seul User

@OneToMany prend deux fonctions callback :

- la première précise l'entité du côté **Many**
- la deuxième précise la propriété inverse qui pointe vers le **One**

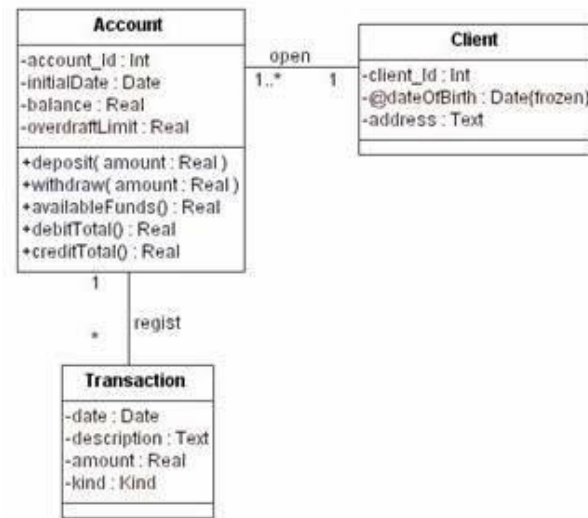
```
@OneToMany( () => Account, (account) => account.user, {  
  cascade: true,}  
accounts: Account[];
```

```
@ManyToOne( () => user, (user) => user.account)
```

```
user: User;
```

Un User possède plusieurs Account,
et chaque Account référence son User via la propriété user.

Relation	Signification
OneToOne	$1 \leftrightarrow 1$
OneToMany	$1 \rightarrow N$
ManyToOne	$N \rightarrow 1$
ManyToMany	$N \leftrightarrow N$



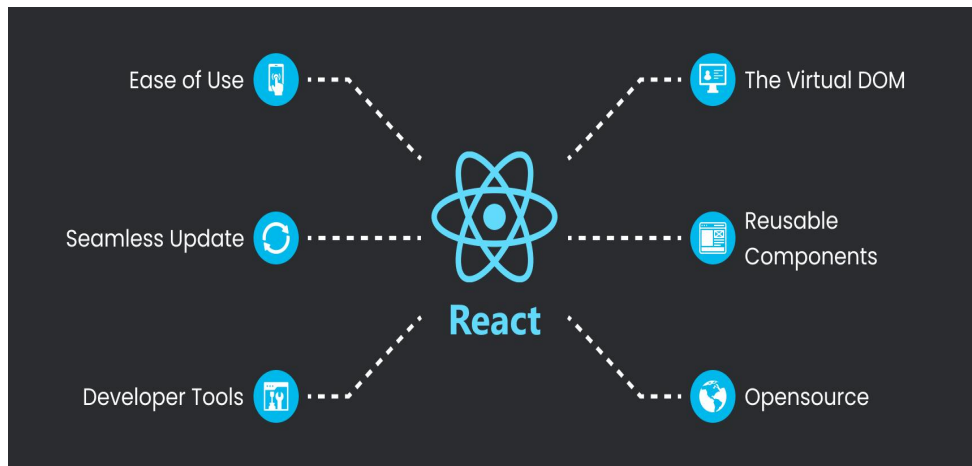
Introduction à React.js

React.js (souvent appelé simplement React) est une bibliothèque **JavaScript** populaire et **open-source** utilisée pour construire des interfaces utilisateur (UI), principalement pour des applications à page unique (SPA - **Single Page Applications**). Développée par Facebook (maintenant Meta), elle est maintenue par Facebook et une large communauté de développeurs. React permet aux développeurs de créer des applications web rapides, évolutives et faciles à maintenir, avec un accent particulier sur le rendu des interfaces utilisateur.

Principales caractéristiques de React.js :

- **Architecture basée sur des composants** : React utilise une architecture basée sur des composants, ce qui signifie que l'interface utilisateur est divisée en petites unités réutilisables appelées composants. Chaque composant contient sa propre logique, son rendu et son état. Cela facilite la gestion et l'évolution des applications.

```
function Greeting() {  
  return <h1>Bonjour, le monde !</h1>;  
}
```



Configuration et Installation d'un Projet React

Prérequis :

- Installer **Node.js**
- Vérifier l'installation avec les commandes suivantes :

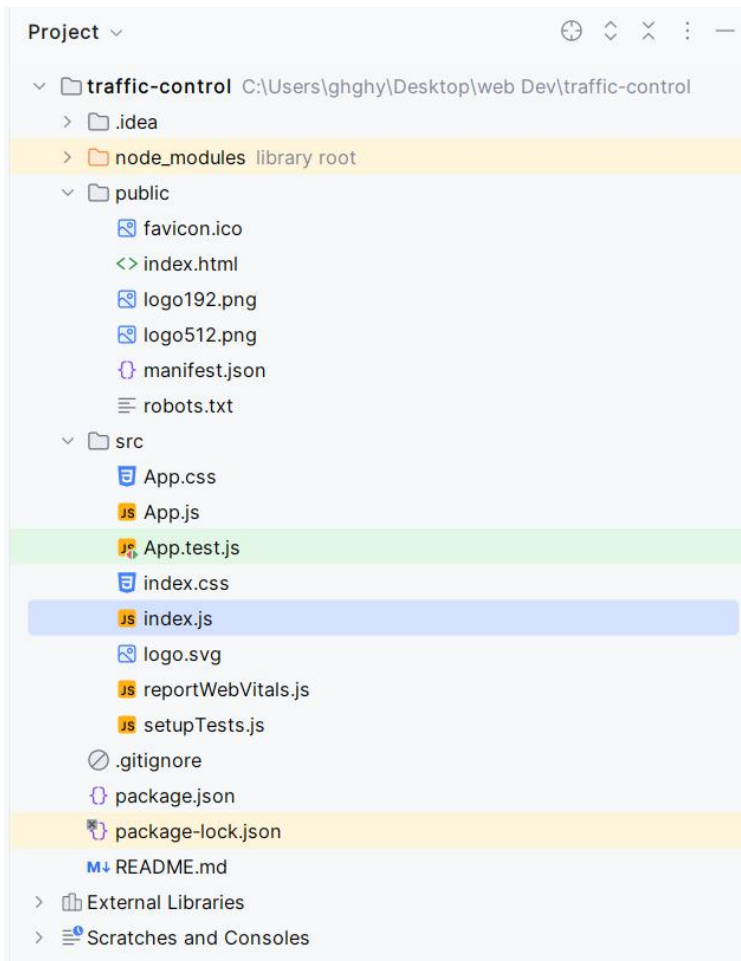
```
node -v  
npm -v
```

Création d'un projet React:

```
npx create-react-app traffic-control  
cd traffic-control  
npm start
```

Structure du projet généré :

- **public/** : Contient index.html et les fichiers statiques.
- **src/** : Contient le code source React.
- **node_modules/** : Bibliothèques installées via npm.
- **package.json** : Liste des dépendances et scripts de gestion du projet.



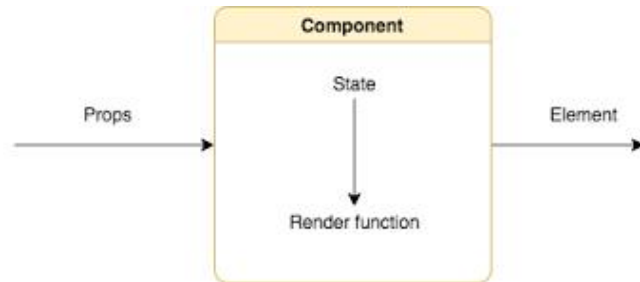
React Component

- Un composant React est une fonction ou une classe JavaScript qui :
- Accepte des **props** (propriétés) en entrée.
- Retourne un **élément** React (écrit généralement en JSX), qui définit la structure de l'interface utilisateur :

- une fonction java script : **Stateless** Component
- une classe héritant de la classe React.Component : **Stateful** Component

Un composant React est défini par :

- l'état du composant (Component State) qui représente le modèle
- le comportement (Behaviour ou Controller)
- Structure de la vue du composant (View)



Functional component

```
function Bonjour(props) {  
  return <h1>Bonjour, {props.nom} !</h1>;  
}
```

Class-based component

```
class Bonjour extends React.Component {  
  render() {  
    return <h1>Bonjour, {this.props.nom} !</h1>;  
  }  
}
```

ReactJS : Java Script XML (JSX)

- JSX (*JavaScript XML*) est une **extension de syntaxe** qui permet d'écrire du code HTML directement dans du JavaScript. Cependant, le navigateur **ne comprend pas JSX**, il doit donc être transformé en JavaScript avant d'être exécuté
- JSX ressemble à du HTML, mais c'est du JavaScript. Les balises comme `<div>` et `<h1>` sont en réalité des éléments React créés en arrière-plan.
- Chaque balise en JSX (comme `<h1>` ou `<div>`) doit être correctement fermée, y compris celles qui se ferment automatiquement comme `<img />`.
- Tous les éléments retournés doivent être contenus dans un seul parent (par exemple, le `<div>` ici).
- App est une fonction qui retourne l'interface utilisateur (UI). Elle est exportée afin de pouvoir être utilisée ailleurs.
- L'importation de React avec **import React from 'react'**; est nécessaire pour utiliser JSX, car JSX est transformé en JavaScript grâce à React.

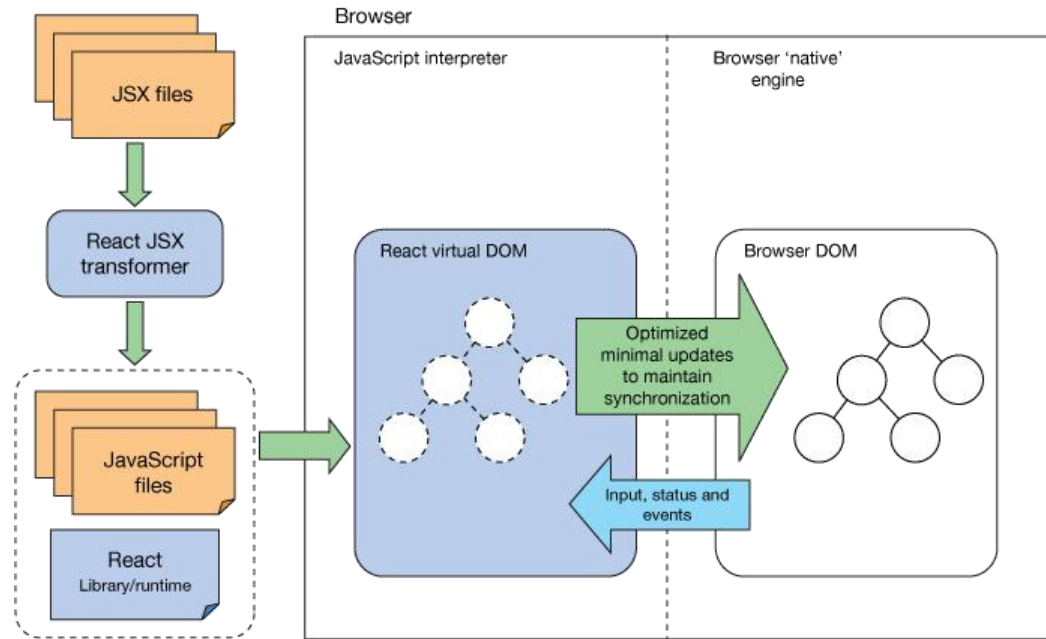
```
import React from 'react';

function App() {
  return (
    <div>
      <h1>Welcome To React Tutorial</h1>
    </div>
  );
}

export default App;
```

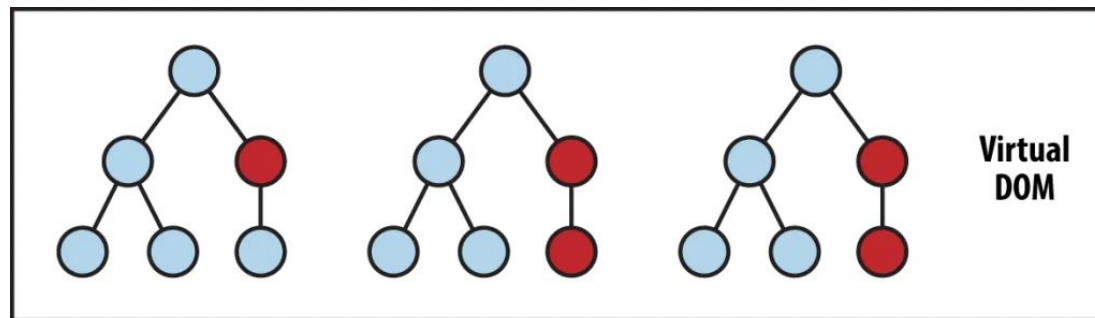
React Component

- Le moteur de rendu JSX est une partie essentielle de React, qui transforme le code JSX en instructions compréhensibles par le moteur JavaScript pour créer et manipuler le DOM (Document Object Model).
- React utilise **Babel**, un transpileur JavaScript, pour convertir JSX en JavaScript standard.
- Le moteur de React génère un DOM virtuel qui sera transformé et synchroniser avec le DOM réel du Navigateur.
- Le Virtual DOM optimise les changements à apporter au niveau du browser DOM. Ce qui lui permet d'accélérer l'aspect réactif du rendu.

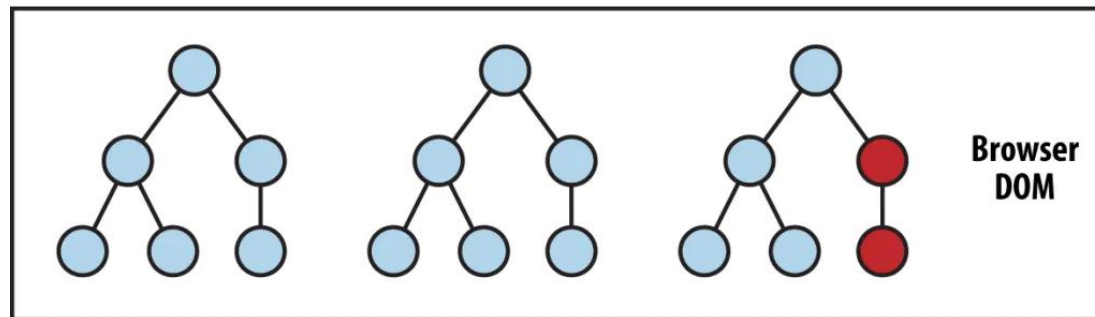


React Components : Virtual DOM

- Lorsqu'un élément change, le navigateur doit recalculer la mise en page (**reflow**) et redessiner (**repaint**) l'ensemble des éléments affectés.
- Cela peut entraîner un effet en cascade, où une modification sur un élément peut impacter plusieurs autres éléments.
- Modifier plusieurs éléments un par un entraîne de nombreux accès au DOM, ce qui ralentit l'application.
- Contrairement à un simple objet JavaScript en mémoire, le DOM est une interface complexe gérée par le navigateur, ce qui le rend plus lent à modifier



State Change → Compute Diff → Re-render



Premier Démo ReactJS : Todo List

Objectif:

Dans cette démo, nous allons créer une application de gestion de tâches ("Todo List") en utilisant ReactJS. L'objectif est de comprendre les concepts fondamentaux de React tels que les composants, l'état (state), les événements, et l'affichage dynamique.

Instructions :

1. Créer un composant TodoApp qui sera le composant principal de l'application.
2. Ajouter un état todos dans le composant principal pour stocker la liste des tâches.
3. Créer un formulaire d'ajout de tâche avec un champ de texte et un bouton pour ajouter une nouvelle tâche à la liste.
4. Afficher la liste des tâches sous forme d'éléments HTML dans une liste non ordonnée.
5. Ajouter un bouton "Supprimer" à chaque tâche pour permettre à l'utilisateur de supprimer une tâche.
6. Implémenter une fonctionnalité pour marquer une tâche comme terminée (en modifiant le style ou en ajoutant une case à cocher).

ReactJS : ES6 array methods

Nouvelles méthodes des tableaux avec ES6

- Avec ES6, de nouvelles méthodes puissantes ont été introduites pour rendre la manipulation des tableaux plus facile et plus fonctionnelle.
- Ces méthodes permettent de modifier, itérer et transformer les tableaux de manière concise et élégante.

map() : Créer un nouveau tableau avec des éléments transformés

- La méthode **map()** crée un nouveau tableau avec les résultats de l'appel d'une fonction pour chaque élément du tableau.
- Ne modifie pas le tableau original

```
const numbers = [1, 2, 3, 4];  
const doubled = numbers.map(num => num *  
2);  
console.log(doubled); // [2, 4, 6, 8]
```

```
const FruitList = [  
  'Banana',  
  'Avocado',  
  'Strawberries',  
  'Orange'  
];  
  
function Examples() {  
  return (  
    <ul>  
      {FruitList.map((fruit, index) => (  
        <li key={index}>{fruit}</li>  
      ))}  
    </ul>  
  );  
}  
export default Examples;
```

ReactJS : ES6 array methods

Nouvelles méthodes des tableaux avec ES6

- Avec ES6, de nouvelles méthodes puissantes ont été introduites pour rendre la manipulation des tableaux plus facile et plus fonctionnelle.
- Ces méthodes permettent de modifier, itérer et transformer les tableaux de manière concise et élégante.

map() : Créer un nouveau tableau avec des éléments transformés

- La méthode **map()** crée un nouveau tableau avec les résultats de l'appel d'une fonction pour chaque élément du tableau.
- Ne modifie pas le tableau original

```
const numbers = [1, 2, 3, 4];  
const doubled = numbers.map(num => num * 2);  
console.log(doubled);  
// [2, 4, 6, 8]
```

filter() : Créer un tableau avec des éléments filtrés

- La méthode **filter()** crée un nouveau tableau avec les éléments qui satisfont la condition fournie.
- Les éléments qui ne passent pas le test sont simplement ignorés.

```
const numbers = [1, 2, 3, 4];  
const evenNumbers = numbers.filter(num => num % 2  
=== 0);  
console.log(evenNumbers); // [2, 4]
```

ReactJS : ES6 array methods

Nouvelles méthodes des tableaux avec ES6

- Avec ES6, de nouvelles méthodes puissantes ont été introduites pour rendre la manipulation des tableaux plus facile et plus fonctionnelle.
- Ces méthodes permettent de modifier, itérer et transformer les tableaux de manière concise et élégante.

splice() : Ajouter ou supprimer des éléments à un tableau

- La méthode **splice()** permet de supprimer, ajouter ou remplacer des éléments dans un tableau.
- Modifie le tableau original et renvoie les éléments supprimés.

array.splice(start, deleteCount, item1, item2, ...)

```
const numbers = [1, 2, 3, 4, 5];
numbers.splice(2, 2, 6, 7);
// À partir de l'index 2, supprime 2 éléments et
// ajoute 6 et 7
console.log(numbers); // [1, 2, 6, 7, 5]
```

concat() : Fusionner deux tableaux

- La méthode **concat()** permet de fusionner plusieurs tableaux en un seul.
- Retourne un nouveau tableau sans modifier les tableaux originaux.

```
const array1 = [1, 2, 3];
const array2 = [4, 5, 6];
const merged = array1.concat(array2);
console.log(merged); // [1, 2, 3, 4, 5, 6]
```

```
const fruits = ['Pomme', 'Banane', 'Cerise'];
console.log(fruits.includes('Banane')); // true
console.log(fruits.includes('Mangue')); // false
```

ReactJS : ES6 array methods

Nouvelles méthodes des tableaux avec ES6

- Avec ES6, de nouvelles méthodes puissantes ont été introduites pour rendre la manipulation des tableaux plus facile et plus fonctionnelle.
- Ces méthodes permettent de modifier, itérer et transformer les tableaux de manière concise et élégante.

splice() : Ajouter ou supprimer des éléments à un tableau

- La méthode **splice()** permet de supprimer, ajouter ou remplacer des éléments dans un tableau.
- Modifie le tableau original et renvoie les éléments supprimés.

array.splice(start, deleteCount, item1, item2, ...)

```
const numbers = [1, 2, 3, 4, 5];
numbers.splice(2, 2, 6, 7);
// À partir de l'index 2, supprime 2 éléments et
// ajoute 6 et 7
console.log(numbers); // [1, 2, 6, 7, 5]
```

concat() : Fusionner deux tableaux

- La méthode **concat()** permet de fusionner plusieurs tableaux en un seul.
- Retourne un nouveau tableau sans modifier les tableaux originaux.

```
const array1 = [1, 2, 3];
const array2 = [4, 5, 6];
const merged = array1.concat(array2);
console.log(merged); // [1, 2, 3, 4, 5, 6]
```

```
const fruits = ['Pomme', 'Banane', 'Cerise'];
console.log(fruits.includes('Banane')); // true
console.log(fruits.includes('Mangue')); // false
```

ReactJS : Class based components

- Utilisent la syntaxe ES6 (class)
- Doivent hériter de React.Component
- Utilisent un constructeur pour gérer l'état (state)
- Accèdent aux props via this.props

Exemple :

Le composant « **Counter** » utilise le **state** interne pour gérer l'état du compteur et reçoit la valeur initiale via les **props**.

```
this.setState({ key1: value1, key2: value2, ... });
```

```
this.setState((prevState, props) => ({ key1: newValue1}));
```

```
class Counter extends Component {
  constructor(props) {
    super(props);
    // Initialisation de l'état local basé sur les props
    this.state = {
      // `initialCount` provient des props
      count: this.props.initialCount,
    };
  }
  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };
  render() {
    return (
      <div>
        <p>Compteur: {this.state.count}</p>
        <button onClick={this.increment}>Incrémenter</button>
      </div>
    );
  }
}
export default Counter;
```

ReactJS : Le Cycle de Vie des Composants

Les composants de classe ont des **méthodes du cycle de vie** qui nous permettent de nous connecter à des étapes spécifiques du cycle de vie du composant.

Montage (Mounting) :

- constructor(): Appelé lors de la création du composant.
- componentDidMount(): Appelé après que le composant soit ajouté au DOM (initialisation).

```
componentDidMount() {  
  console.log("Composant monté !");  
}
```

Mise à Jour (Updating) :

- shouldComponentUpdate(): Détermine si le composant doit se ré-render lorsque son état ou ses props changent.
- componentDidUpdate(): Appelé après que le composant ait été mis à jour (re-rendu).

```
componentDidUpdate(prevProps, prevState) {  
  console.log("Composant mis à jour !");  
}
```


ReactJS : Le Cycle de Vie des Composants

Démontée (Unmounting) :

- **componentWillUnmount()**: Appelé juste avant que le composant ne soit retiré du DOM. Utile pour effectuer des nettoyages.

```
componentWillUnmount() {  
  console.log("Composant démonté !");  
}
```

Demo pratique

ReactJS : Les Composants Fonctionnels

Composant Fonctionnel : Un composant fonctionnel est une fonction JavaScript qui prend des props en entrée et retourne des éléments React.

```
import React from 'react';

const MyComponent = (props) => {
  return <div>Hello, {props.name}!</div>;
};

export default MyComponent;
```

ReactJS : Le Cycle de Vie dans CF

Depuis React 16.8, les **Hooks** permettent aux composants fonctionnels de gérer leur état et d'utiliser des effets secondaires, comme les composants de classe. Cela remplace la gestion du cycle de vie dans les composants fonctionnels.

Les Hooks principaux :

- **useState** : Gère l'état dans les composants fonctionnels.
- **useEffect** : Gère les effets secondaires, remplaçant des méthodes comme **componentDidMount**, **componentDidUpdate**, et **componentWillUnmount**.

Mounting (Montage):

useEffect avec tableau de dépendances vide [] :
Agit comme **componentDidMount**, appelé une fois après le montage.

```
const ExampleComponent = () => {  
  const [count, setCount] = useState(0);  
  // L'équivalent de componentDidMount  
  useEffect(() => {  
    console.log('Composant monté !');  
  },  
  []); // Tableau vide signifie que cela ne se produit  
  qu'une fois (montée)  
  
  return (  
    <div>  
    </div>  
  );  
};
```

ReactJS : Le Cycle de Vie dans CF

Updating (Mise à jour)

useEffect avec dépendances : Agit comme componentDidMount. Il se déclenche après chaque mise à jour où les dépendances spécifiées changent.

```
useEffect(() => {  
  console.log('Le compteur a changé');  
  document.title = `Compteur: ${count}`;  
}, [count]); // Se déclenche lorsque `count` change
```

Unmounting (Démontage)

Retour de la fonction de nettoyage dans useEffect : Agit comme componentWillUnmount, permettant de nettoyer les effets secondaires.

```
useEffect(() => {  
  console.log('Composant monté !');  
  // Retour de la fonction de nettoyage (équivalent de componentWillUnmount)  
  return () => {  
    console.log('Composant démonté !');  
  };  
}, []); // Cela se déclenche une seule fois, au démontage
```

```
componentDidMount    ⇔ useEffect(() => {...}, [])  
componentDidUpdate   ⇔ useEffect(() => {...}, [dep])  
componentWillUnmount ⇔ useEffect(() => { return () => {...}; }, [])
```

ReactJS : useRef Hook

Le hook **useRef** est utilisé pour garder une référence mutable à un élément du DOM ou une valeur qui persiste à travers les re-rendus.

Contrairement à **useState**, **useRef** ne provoque pas un re-rendu lorsqu'il est mis à jour.

Utilisations principales :

1. Accéder à un élément DOM (par exemple, un input ou une image).
2. Stocker une valeur persistante sans déclencher de re-rendu (par exemple, une valeur de compteur).

```
const ref = useRef(initialValue);
```

- **initialValue**: La valeur initiale de la référence (peut être null ou toute autre valeur).

- **ref.current**: C'est la propriété qui contient la référence. Elle est modifiable sans provoquer un re-rendu.

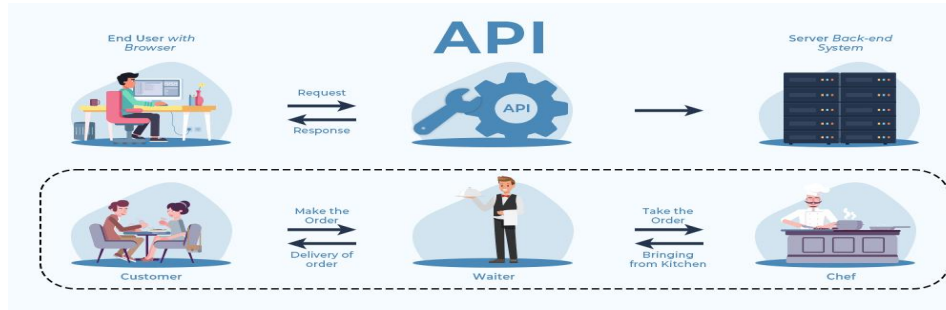
```
function FocusInput() {  
  const inputRef = useRef(null);  
  const handleFocus = () => {  
    inputRef.current.focus(); // Utilisation de useRef pour donner le focus à l'input  
  };  
  return (  
    <div>  
      <input ref={inputRef} type="text" placeholder="Cliquez sur le bouton pour focus" />  
      <button onClick={handleFocus}>Donner le focus à l'input</button>  
    </div>  
  );  
}
```

Introduction aux APIs

Application Programming Interface (API) est un ensemble de règles et de protocoles permettant à différents logiciels ou applications de communiquer entre eux. Elle définit les méthodes et les outils que les programmeurs peuvent utiliser pour intégrer des fonctionnalités spécifiques dans leurs propres applications sans avoir à recréer ces fonctionnalités à partir de zéro.



- Une **API** sert de **pont** entre différentes applications ou services, permettant d'envoyer des **requêtes**, d'obtenir des **réponses** et d'interagir de manière fluide avec d'autres systèmes.



- Les **API** sont couramment utilisées dans le développement d'applications mobiles, web, et de systèmes logiciels afin de simplifier les tâches et d'améliorer l'**interopérabilité** entre différents systèmes.

Pourquoi le développement des APIs ?

Développent des APIs ?

Permettent à un logiciel d'exposer ses fonctionnalités pour être utilisées par d'autres logiciels(ex : Maps API, Météo API, etc.)

Facilitent la réutilisation de la logique applicative pour créer différents logiciels personnalisés (Ex: PayPal).

Permettent de réutiliser la logique applicative sur différentes plateformes.

Facilitent la communication entre le client et le serveur

Permettent de tester les applications plus rapidement, avec plus de précision.

Les types d'APIs

1. API Web : Est une interface accessible via HTTP/HTTPS qui permet à différentes applications de communiquer sur Internet. Ces API sont généralement utilisées pour interagir avec des services externes ou des systèmes distants, souvent en utilisant des formats de données comme **JSON** ou **XML**.

Exemple :

Imaginons une API Web qui vous permet de récupérer des informations météorologiques pour une ville donnée.

URL de l'API : <https://api.openweathermap.org/data/2.5/weather>

Méthode HTTP : GET

Exemple de requête : **GET** https://api.openweathermap.org/data/2.5/weather?q=Casablanca,MA&appid=YOUR_API_KEY

Response :

```
{
  "weather": [
    {
      "main": "Clear",
      "description": "clear sky"
    }
  ],
  "main": {
    "temp": 289.92,
    "pressure": 1010,
    "humidity": 76
  },
  "name": "Paris"
}
```


Les types d'APIs

2. API Locales : est une API qui fonctionne sur le même système ou serveur que l'application qui la consomme. Elle permet à une application de communiquer avec elle-même ou avec d'autres services locaux sur la même machine ou dans un environnement contrôlé. Ces APIs sont souvent utilisées dans des applications de bureau ou dans des systèmes embarqués.

Exemple :

Imaginons une application de gestion des feux de signalisation dans une ville marocaine, par exemple à Rabat, qui expose une **API locale** pour interagir avec le système de feux de circulation.

URL de l'API : <http://localhost:3000/api/traffic-lights>

Méthode HTTP : **POST**

Exemple de requête :

```
POST http://localhost:3000/api/traffic-lights
Body : {
  "intersection": "Avenue Mohammed V & Rue Al Quods",
  "status": "green"
}
```

Réponse JSON :

```
{
  "message": "Traffic light updated successfully",
  "status": "green"
}
```

On distingue plusieurs types d'API Web selon leur architecture et leur mode de communication.

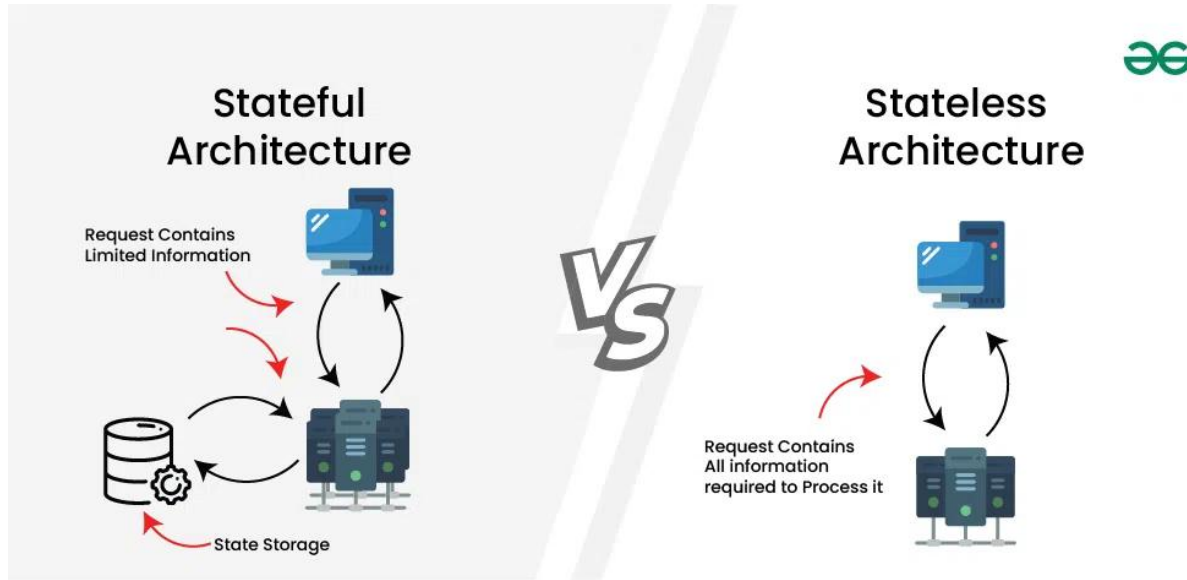
1. **API REST (Representational State Transfer)**: est un type d'API web qui suit les principes de l'architecture REST. Elle permet aux applications de communiquer entre elles via des requêtes HTTP en utilisant des **endpoints** (URL spécifiques) pour accéder et manipuler des ressources.

Principes fondamentaux de REST:

- **Stateless** : Chaque requête est indépendante, le serveur ne conserve pas l'état du client.
- **Utilisation le Protocol HTTP** : REST utilise les méthodes standards du protocole HTTP :
- **GET** : récupérer des données
- **POST** : créer une nouvelle ressource
- **PUT** : mettre à jour une ressource existante
- **DELETE** : supprimer une ressource
- Format des données : REST utilise principalement **JSON** (mais peut aussi supporter **XML**).
- Architecture basée sur les ressources : Chaque ressource est identifiée par une URL unique.

Stateful vs Stateless Architecture

Dans une **architecture stateful**, le serveur conserve une **trace** de l'état ou des informations de **session** de chaque client pendant toute la durée de la communication. Cela permet au serveur de se souvenir des requêtes précédentes, de maintenir des sessions utilisateur et de suivre l'état de l'application au fil du temps. Les informations de session peuvent être stockées dans la mémoire du serveur, une base de données, ou d'autres mécanismes persistants.



Dans une architecture **Stateless**, le serveur **ne stocke aucune** information de session client entre les requêtes. Chaque requête envoyée par le client est traitée comme une transaction indépendante, sans dépendance aux interactions précédentes. Les architectures **Stateless** sont conçues pour être plus évolutives et tolérantes aux pannes, car elles ne nécessitent pas de ressources serveur pour conserver l'état du client.

Rest API

Les **REST APIs** respectent la notion de **stateless architecture** en appliquant les principes suivants :

- **Aucune session stockée côté serveur** : le serveur ne conserve aucune information sur l'état du client entre les requêtes. Chaque requête doit contenir toutes les informations nécessaires pour être traitée indépendamment.

Exemple:

```
GET /users/123  
Authorization: Bearer <token>
```

- **Chaque requête est indépendante** : Les requêtes envoyées au serveur REST doivent être autonomes. Cela signifie que le serveur ne se souvient pas des requêtes précédentes et ne stocke pas d'état temporaire entre elles.

Exemple:

```
GET /orders?userId=123  
Authorization: Bearer <token>
```

- **Les données de session sont gérées côté client** : Dans une architecture REST **stateless**, si un état ou une session est nécessaire, c'est au client de le gérer et de l'envoyer à chaque requête.

Exemple:

```
POST /checkout  
Content-Type: application/json  
Authorization: Bearer <token>  
{  
  "cart": [  
    { "productId": 1, "quantity": 2 },  
    { "productId": 5, "quantity": 1 }  
  ]  
}
```

Introduction aux API Web et Fetch API

fetch est une fonction native de JavaScript permettant de faire des requêtes HTTP (GET, POST, PUT, DELETE, etc.) pour récupérer ou envoyer des données entre le client (navigateur) et un serveur. **fetch** est basée sur des promesses et remplace l'ancienne méthode **XMLHttpRequest**.

Pourquoi utiliser fetch ?

- Plus moderne et plus simple à utiliser que XMLHttpRequest.
- Supporte les promesses.
- Permet de récupérer facilement des données à partir d'API REST.
- Prise en charge nativement dans tous les navigateurs modernes.

```
fetch(url, options)
  .then(response => response.json()) // Convertit la réponse en JSON
  .then(data => console.log(data))    // Utilise les données
  .catch(error => console.error('Erreur:', error)); // Gère les erreurs
```

url : L'URL à laquelle envoyer la requête (API, fichier, etc.).

options : Paramètres de la requête (méthode, en-têtes, corps de la requête, etc.) – optionnel.

Introduction aux API Web et Fetch API

Exemple Simple d'utilisation de fetch

```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) { // Vérifie si la réponse est correcte
      throw new Error('Erreur de réseau');
    }
    return response.json(); // Convertir en JSON
  })
  .then(data => console.log(data)) // Afficher les données
  .catch(error => console.error('Erreur:', error)); // Capturer les
erreurs
```